Noah Jutz

Algorithmen und Datenstrukturen

Notizen

1 Mathematische Grundlagen 2
1.1 Fibonacci-Folge 2
1.2 Charakteristisches Polynom 2
1.3 Strukturelle Induktion 3
2 Komplexität 3
2.1 O-Notation 3
2.2 Komplexitätsklassen 3
2.3 WC, BC, AC 4
2.4 Rekursionsgleichungen 4
3 Einfache Algorithmen 5
3.1 ggT 5
3.2 kgV 6
3.3 Sieb des Eratosthenes 7
3.4 Addition um 1 8
3.5 MaxTeilSum 8
3.6 Türme von Hanoi 10
3.7 Rod Cutting Problem 10
4 Sortieralgorithmen 11
4.1 Insertion Sort 11
4.2 Bubble Sort 13
4.3 Selection Sort 14
4.4 Quicksort 14
4.5 Merge Sort 16
4.6 Heap Sort 17
4.7 Counting Sort 19
4.8 Bucket Sort 19
4.9 Map Sort 20
5 Suchbäume 20
5.1 Binärer Suchbaum 20
5.2 AVL-Baum 21
5.3 RS-Baum 21
5.4 B-Baum 21
6 Datenstrukturen 22
6.1 Hashtabelle 22
6.2 Skip-Liste 22
7 Graphen 23
7.1 Speicherung 23
7.2 Durchlaufen 23

1 Mathematische Grundlagen

1.1 Fibonacci-Folge

Eine *Differenzengleichung* der Ordnung k definiert jedes Glied x_n einer Folge in Abhängigkeit der Vorgänger $x_{n-1}, x_{n-2}, \ldots, x_{n-k}$. Es müssen k Initialwerte definiert werden, sodass die Rekursion beendet werden kann.

$$x_n = f(n, x_{n-1}, x_{n-2}, \ldots, x_{n-k})$$

Eine *lineare* Differenzengleichung fordert, dass die Funktion ein lineares Polynom ist.

$$x_n = a_1 x_{n-1} + a_2 x_{n-2} + \ldots + a_n x_{n-k} + b$$

Man nennt man diese *homogen*, wenn sie keinen konstanten Term b hat.

Definition

Die Fibonacci-Folge erfüllt eine *homogene lineare Differenzengleichung* der Ordnung $k = 2$.

$$x_n = a_1 x_{n-1} + a_2 x_{n-2}$$

Die Koeffizienten sind definiert als $a_1 = a_2 = 1$, und die Initialwerte als $x_0 = 0$ und $x_1 = 1$. Die Fibonacci-Folge ist also definiert als:

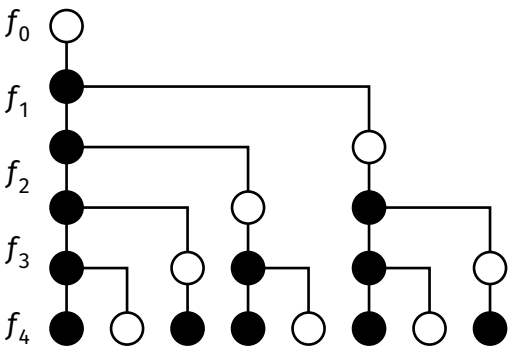
$$f_n = \begin{cases} 0 \text{ falls } n = 0 \\ 1 \text{ falls } n = 1 \\ f_{n-1} + f_{n-2} \text{ sonst} \end{cases}$$

Die ersten 10 Fibonacci-Zahlen sind:

f_0	f_1	f_2	f_3	f_4	f_5	f_6	f_7	f_8	f_9
0	1	1	2	3	5	8	13	21	34

Fibonaccis Kaninchen

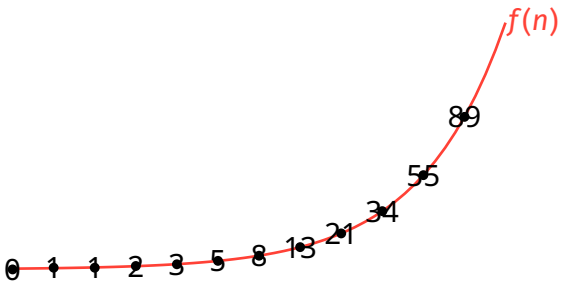
Wie viele Kaninchen gibt es nach n Monaten, wenn jedes erwachsene Kaninchen (mindestens 1 Monat alt) jeden Monat ein Nachkommen produziert, und niemals stirbt?



Die Anzahl an Kaninchen in einem Monat n sind zunächst alle Kaninchen, die es im Monat davor schon gibt (f_{n-1}). Es kommen f_{n-2} neue Kaninchen dazu, weil jedes erwachsene Kaninchen ein neues zeugt. Um erwachsen zu sein, muss es schon zwei Monate zuvor existiert haben. Daraus ergibt sich die Fibonacci-Folge. [1]

$$f_n = f_{n-1} + f_{n-2}$$

Iterative Lösung



Es gibt eine Funktion $f(n)$, welche die n-te Fibonacci-Zahl ohne Rekursion berechnet. Für diese Funktion muss gelten:

$$f(n) = f(n-1) + f(n-2)$$

Das Verhältnis zwischen zwei Fibonacci-Zahlen $\frac{f_n}{f_{n-1}}$ konvergiert für $n \to \infty$ gegen einen konstanten Wert $\varphi \approx 1.618$. Beweis: Abschnitt 1.2. Die Fibonacci-Folge nähert sich einer geometrischen Folge an.

$$x_n = x_0 \cdot \varphi^n$$

1.2 Charakteristisches Polynom

Jede quadratische Matrix A hat genau ein charakteristisches Polynom [2].

$p_A(\lambda) = \det(A - \lambda I)$

Die Nullstellen dieses Polynoms sind die Eigenwerte von A. Die Eigenvektoren mit Eigenwert λ sind die Lösung der Gleichung

$(A - \lambda I) \cdot \vec{v} = \vec{0}$

1.3 Strukturelle Induktion

Verallgemeinerung der vollständigen Induktion; siehe Wikipedia.

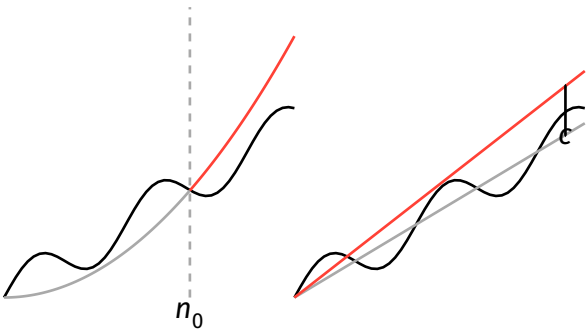
2 Komplexität

2.1 O-Notation

Ein Landau-Symbol $M \in \{O, o, \Omega, \omega, \Theta\}$ ist eine Menge, welche aus Funktionen besteht. Diese Menge ist in Abhängigkeit einer Funktion f definiert, welche alle $g \in M$ nach oben oder unten beschränkt.

O	$g(n) \le c \cdot f(n)$	Obere Schranke
o	$g(n) < c \cdot f(n)$	
Ω	$g(n) \ge c \cdot f(n)$	Untere Schranke
ω	$g(n) > c \cdot f(n)$	
Θ	$c_1 \cdot f(n) \le g(n) \le c_2 \cdot f(n)$	Beides

Die Schranke muss erst ab einem beliebigen $n_0 \in \mathbb{N}$ gelten, und sie darf um einen Konstanten Faktor $c > 0$ von f abweichen.



Grenzwertdefinition

$$\lim_{n\to\infty} \frac{g(n)}{f(n)} = \begin{cases} \infty \Leftrightarrow \begin{cases} f(n) \in O(g(n)) \\ g(n) \in \Omega(f(n)) \end{cases} \\ c \Leftrightarrow \begin{cases} g(n) \in \Theta(f(n)) \\ f(n) \in \Theta(g(n)) \end{cases} \\ 0 \Leftrightarrow \begin{cases} f(n) \in \Omega(g(n)) \\ g(n) \in O(f(n)) \end{cases} \end{cases}$$

Mengendefinition

$$O(f(n)) = \left\{ g(n) \;\middle|\; \begin{array}{l} \exists n_0 \in \mathbb{N}, c > 0 : \\ \forall n \ge n_0 : g(n) \le c \cdot f(n) \end{array} \right\}$$

$$\Omega(f(n)) = \left\{ g(n) \;\middle|\; \begin{array}{l} \exists n_0 \in \mathbb{N}, c > 0 : \\ \forall n \ge n_0 : g(n) \ge c \cdot f(n) \end{array} \right\}$$

$$\Theta(f(n)) = \left\{ g(n) \;\middle|\; \begin{array}{l} \exists n_0 \in \mathbb{N}, c > 0 : \\ \forall n \ge n_0 : g(n) = c \cdot f(n) \end{array} \right\}$$

Rechenregeln

- $f \in \Theta(f)$
- $g \in \Theta(f)$
  $\Rightarrow c \cdot g \in \Theta(f)$
- $g_1 \in \Theta(f_1), g_2 \in \Theta(f_2)$
  $\Rightarrow g_1 + g_2 \in O(\max(f_1, f_2))$
  $\Rightarrow g_1 + g_2 \in \Omega(\min(f_1, f_2))$
  $\Rightarrow g_1 + g_2 \in \Theta(f_1 + f_2)$
  $\Rightarrow g_1 \cdot g_2 \in \Theta(f_1 \cdot f_2)$

2.2 Komplexitätsklassen

- $\Theta(1)$

  Diese Laufzeitfunktionen konvergieren gegen einen konstanten Wert. Es gibt keine geringere Komplexität als $\Theta(1)$.

- $\Theta(\log n)$

  Ein Logarithmus zur Basis b_1 kann mit einem konstanten Faktor die Basis b_2 verwenden.

$$\log_{b_1} n = \underbrace{\log_{b_1} b_2}_{\text{konstant}} \cdot \log_{b_2} n$$

2.3 WC, BC, AC

Ein Algorithmus hat bei unterschiedlichen Eingaben Σ^n der gleichen Länge n ggf. unterschiedliche Laufzeiten.

Beispiel

Sei A = `inc_bin`. Desto mehr 1en am Ende hängen, desto länger muss der Carry weitergereicht werden.

$T_A(\texttt{0000}) < T_A(\texttt{0111})$

Worst-Case und Best-Case

Die Menge aller Laufzeitfunktionen bei einer Eingabelänge n sei

$$\Phi(n) \coloneqq \{T(x) \mid x \in \Sigma^n\}$$

Die Worst- und Best-Case Laufzeiten sind jeweils:

$$\begin{aligned} T^{\text{WC}}(n) &= \max(\Phi(n)) & T^{\text{BC}}(n) &= \min(\Phi(n)) \\ &= \max_{x \in \Sigma^n}\{T(x)\} & &= \min_{x \in \Sigma^n}\{T(x)\} \end{aligned}$$

Die Funktionen in Φ sind nach Wachstumsrate geordnet. Es gilt $g \leq f$, falls f dominiert:

$$\lim_{n \to \infty} \frac{g(n)}{f(n)} \leq c$$

Die größte Funktion T^{WC} beschränkt alle $T \in \Phi$ nach oben, die kleinste Funktion T^{BC} nach unten.

$$T(n) = O\left(T^{\text{WC}}(n)\right) \qquad T(n) = \Omega\left(T^{\text{BC}}(n)\right)$$

Average-Case

Die Average-Case-Laufzeit ist das arithmetische Mittel über alle Laufzeiten.

$$T^{\text{AC}}(n) = \frac{1}{|\Sigma^n|} \sum_{x \in \Sigma^n} T(x)$$

Beispiel: Abschnitt 2.1

2.4 Rekursionsgleichungen

Die Komplexität rekursiver Algorithmen wird durch eine Rekursionsgleichung beschrieben.

Aufstellen der Rekursionsgleichung

Eine rekursive Funktion hat mindestens eine Abbruchbedingung, welche bei einer trivialen Eingabelänge n eintritt. Die Laufzeiten der Base-Cases sind die *Initialwerte* der Rekursionsgleichung.

$$T(0) = \ldots, \quad T(1) = \ldots, \quad T(2) = \ldots, \quad \ldots$$

Für alle komplexeren Eingaben n gibt es mindestens einen Selbstaufruf. Dieser Selbstaufruf löst ein einfacheres Subproblem, hat also eine geringere Eingabelänge. Beispiel:

$$T(n) = 5T(n-1) + T\left(\sqrt{n}\right) + n^2$$

Charakteristisches Polynom

Lineare Rekursionsgleichungen können als exponentielle Funktion dargestellt werden.

$$T(n) = \sum_{i=1}^{k} a_i \cdot T(n-i) \implies T(n) = r^n$$

Um die Basis r herauszufinden, lösen wir das charakteristische Polynom.

$$r^n = \sum_{i=0}^{k} a_i \cdot r^{n-i}$$

Substitutionsmethode

Komplexität abschätzen und mittels vollständiger Induktion beweisen.

Iterationsmethode

Schrittweise die Formel in Selbstaufrufe einsetzen und ein Muster erkennen.

Master-Theorem

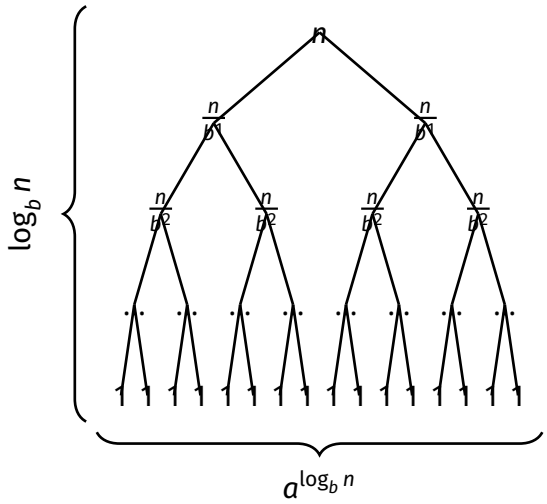
Viele Divide & Conquer Algorithmen zerlegen die Eingabe in b Teile, und erzeugen a

Subprobleme, welche diese Teile verarbeiten. Die Ergebnisse der Subprobleme werden anschließend kombiniert. Die Laufzeit der Zerlegung und Kombination in jedem Rekursionsschritt ist $f(n)$.

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

Der Einfachheit halber gehen wir von einem Base-Case $T(1) = \Theta(1)$ aus.

Desto größer a ist, desto breiter wird der Rekursionsbaum, und desto kleiner b ist, desto tiefer wird die Rekursion. Der Rekursionsbaum sieht bei $a = 2$ so aus:



Die Gesamtlaufzeit setzt sich aus den *rekursiven Schritten* und den *Base Cases* zusammen.

$$T(n) = \underbrace{\sum_{k=0}^{\log_b n - 1} a^k \cdot f\left(\frac{n}{b^k}\right)}_{=:T_R} + \underbrace{a^{\log_b n} \cdot \Theta(1)}_{=:T_B}$$

Ausschlaggebend für die asymptotische Laufzeit ist, ob T_R oder T_B dominiert.

$T_R(n) \leq a^{\log_b n} \cdot f(n)$

$T_R(n) \geq f(n)$

$a^{\log_b n} = n^{\log_b a}$

3 Einfache Algorithmen

3.1 ggT

Man sagt „x teilt a“, wenn a ohne Rest durch x teilbar ist.

$x \mid a$

Der größte gemeinsame Teiler von a und b $\gcd(a, b)$ ist der größte Wert, für den gilt:

$x \mid a \wedge x \mid b$

Primfaktorzerlegung

Um den ggT von a und b zu ermitteln, können wir a und b probeweise durch Faktoren teilen. Beispiel mit $a = 3528, b = 3780$:

	a	b	ggT	
÷ 2	528	3780	1	· 2
÷ 2	1764	1890	2	· 2
÷ 9	882	945	4	· 9
...	98	105	45	...
	14	15	252	

Um systematischer vorzugehen, ermitteln wir die gemeinsamen minimalen Faktoren von a und b durch Primfaktorzerlegung.

$3528 = 2^3 \cdot 3^2 \cdot 5^0 \cdot 7^2$

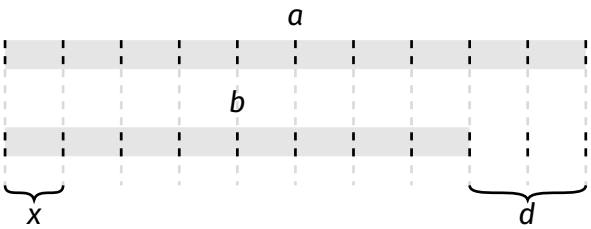
$3780 = 2^2 \cdot 3^3 \cdot 5^1 \cdot 7^1$

Die gemeinsamen Faktoren von a und b sind in diesem Beispiel $2^2, 3^2, 7$. Multipliziert man sie, so erhält man

$\gcd(3528, 3780) = 2^2 \cdot 3^2 \cdot 7 = 252$

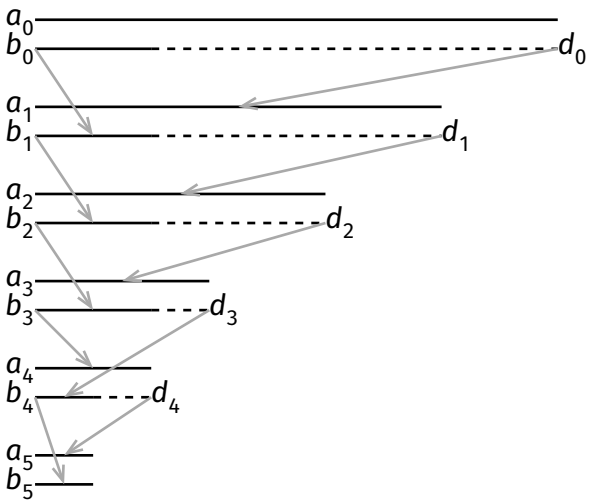
Euklid

Haben a und b mit $a > b$ einen gemeinsamen Teiler x, so ist die Differenz $d = a - b$ auch ein vielfaches von x.



Die Rechnung ist also gleich, wenn statt a und b die Parameter b und d verwendet werden.

$\gcd(a, b) = \gcd(a - b, b) = x$



Irgendwann sind a_n und b_n gleich. Der ggT ist dann $x = a_n = b_n$. Dieses x teilt auch a_0 und b_0, weil der umgekehrte Schritt lediglich ein vielfaches von x zu a addiert.

$a_{n-1} = a_n + b_n \qquad b_{n-1} = b_n$

Dieser Algorithmus zur berechnung des ggT ist der **einfache euklidische Algorithmus**. Der Pseudo-Code ist wie folgt.

```
def gcd_euclid(a, b):
  if a == b: return a
  a_next = max(b, a - b)
  b_next = min(b, a - b)
  gcd(a_next, b_next)
```

Passt ein b mehrmals in ein a, so müssen mehrere Subtraktionsschritte getätigt werden, welche durch eine Moduloberechnung konsolidiert werden können.

$$a_0 \xrightarrow{-b} a_1 \xrightarrow{-b} a_2 \xrightarrow{-b} a_n$$

$$a_0 \overset{\bmod b}{\dashrightarrow} a_n$$

Der **verbesserte Algorithmus** gibt b aus, wenn $a \bmod b = 0$, weil b der kleinere Wert ist und a durch b teilbar ist.

Des Weiteren ist $r = a \bmod b$ per Definition des Modulo immer kleiner als b, wir können uns also die max/min-berechnung sparen.

```
def gcd_euclid_fast(a, b):
    if a % b == 0: return b
    a_next = b
    b_next = a % b
    return gcd(a_next, b_next)
```

Iterativ kann der Algorithmus folgendermaßen umgeschrieben werden:

```
def gcd_euclid_fast_iterative(a, b):
  while a % b != 0:
    a, b = b, a % b
  return b
```

3.2 kgV

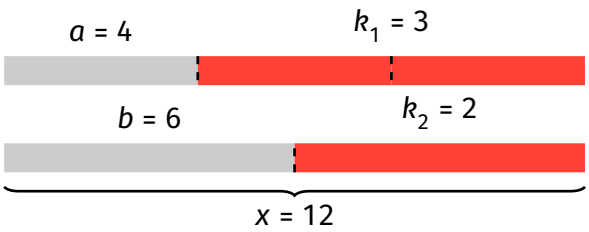
Man nennt x ein Vielfaches von a, wenn es ein $k \in \mathbb{Z}$ gibt, sodass

$x = k \cdot a$

Ein gemeinsames Vielfaches von a und b erfüllt mit $k_1, k_2 \in \mathbb{Z}$:

$x = k_1 \cdot a = k_2 \cdot b$

Zum Beispiel ist 12 ein Vielfaches von 4 und 6, weil $12 = 3 \cdot 4 = 2 \cdot 6$.



Das kleinste gemeinsame Vielfache ist der kleinste Wert x, der sowohl ein Vielfaches von a als auch von b ist. Man schreibt:

$\operatorname{lcm}(a, b)$

Eine naive Implementierung prüft alle $x \in \mathbb{N}$ auf die Bedingung $a \mid x \wedge b \mid x$.

```
def lcm_naive(a, b):
  x = 1
  while x % a ≠ 0 or x % b ≠ 0:
    x += 1
  return x
```

Primfaktorzerlegung

Um das kleinste gemeinsame Vielfache von a und b zu ermitteln, nehmen wir dieses mal nicht die Schnittmenge beider Primfaktorzerlegungen, sondern die Vereinigung. Wir erweitern also die eine Zerlegung um die Faktoren, welche zusätzlich in der anderen sind.

$$\begin{array}{ccccccccc} 3528 & = & 2^3 & \cdot & 3^2 & \cdot & 5^0 & \cdot & 7^2 \\ & & \downarrow \cdot 2 & & \uparrow \cdot 3 & & \uparrow \cdot 5 & & \downarrow \cdot 7 \\ 3780 & = & 2^2 & \cdot & 3^3 & \cdot & 5^1 & \cdot & 7^1 \end{array}$$

Zusammenhang ggT / kgV

Wir haben etabliert, dass beim ggT die gemeinsamen Primfaktoren multipliziert werden, während beim kgV die überschüssigen Primfaktoren hinzumultipliziert werden.

$$\gcd(a, b) = \prod_{p \in \mathbb{P}} p^{\min(e_p, f_p)}$$

$$\operatorname{lcm}(a, b) = \prod_{p \in \mathbb{P}} p^{\max(e_p, f_p)}$$

Daraus lässt sich folgender Zusammenhang schließen:

$$\gcd(a, b) \cdot \operatorname{lcm}(a, b) = a \cdot b$$

Das liegt daran, dass:

$$p^{\min(e_p, f_p)} \cdot p^{\max(e_p, f_p)} = p^{e_p + f_p}$$

3.3 Sieb des Eratosthenes

Eine Primzahl hat genau zwei Teiler: sich selbst und 1. Das Gegenstück ist eine Zusammengesetzte Zahl; diese hat mindestens zwei Primfaktoren. Die 1 ist weder prim noch zusammengesetzt.

$\mathbb{P} = \{2, 3, 5, 7, 11, 13, 17, 19, \ldots\}$

Die Menge aller Primzahlen sind also die natürlichen Zahlen ohne zusammengesetze Zahlen und ohne 1.

Algorithmus

Um alle Primzahlen bis zu einem Wert n herauszufinden, entfernen wir schrittweise alle zusammengesetzten Zahlen aus der Menge {2, 3, 4, ..., n}. Im Beispiel ist $n = 50$.

	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48	49	50

In jeder Iteration des Algorithmus wählen wir die kleinste Zahl p, welche noch nicht markiert wurde. Dann werden alle Vielfachen von p gestrichen und p als Primzahl markiert.

Der erste Schritt ist trivial: Die kleinste Primzahl in $\mathbb{P}$ ist 2.

	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48	49	50

Die nächste nicht gestrichene Zahl ist 3. Sie ist kein Vielfaches von 2, und auch kein Vielfaches von irgend einer anderen Primzahl, weil es keine Primzahl kleiner als 2 gibt. Daraus folgt, dass 3 eine Primzahl ist.

	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48	49	50

Der nächste Wert ist 5. Er ist nicht Vielfaches von irgend einer Zahl kleiner als sich selbst (außer 1), daher ist er auch eine Primzahl.

	2	**3**	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48	49	50

Das kleinste noch nicht gestrichene Vielfache einer Primzahl p ist jeweils p^2. Das liegt daran, dass jedes Vielfache von p auch ein Vielfaches von k ist.

$k \cdot p = p \cdot k$

Zum Beispiel ist 15 ein Vielfaches von $p = 5$ mit $k = 3$, aber wurde davor schon als Vielfaches von $p = 3$ mit $k = 5$ gestrichen.

	2	**3**	4	**5**	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48	49	50

Der Algorithmus ist beendet, wenn $p^2 > n$, weil es dann kein Vielfaches von p gibt, welches nicht davor schon gestrichen wurde. Damit wurden alle zusammengesetzten Zahlen gestrichen, und es bleiben nur noch Primzahlen.

	2	**3**	4	**5**	6	**7**	8	9	10
11	12	**13**	14	15	16	**17**	18	**19**	20
21	22	**23**	24	25	26	27	28	**29**	30
31	32	33	34	35	36	**37**	38	39	40
41	42	**43**	44	45	46	**47**	48	49	50

3.4 Addition um 1

Ein Stellenwertsystem mit Basis b besteht aus b Schriftzeichen mit jeweils einem Wert aus $\{0, 1, 2, ..., b - 1\}$. Der Stellenwert des Vorgängers wird um Faktor b erhöht.

...	b^3	b^2	b^1	b^0

Der Zahlenwert an Stelle i ist das Produkt Des Stellenwertes b^i und des Ziffernwertes a_i. Der Zahlenwert einer Ziffernfolge $a_n ... a_2 a_1 a_0$ ist die Summe

$$Z = \sum_{i=0}^{n} a_i \cdot b^i$$

Es gibt für jede natürliche Zahl genau eine b-adische Darstellung. Beweis: Abschnitt 1.1

Algorithmus

Aus dem induktiven Beweis ergibt sich auch der Algorithmus zur Addition um 1 (bei diesem vereinfachten Algorithmus ist ein Integer Overflow möglich).

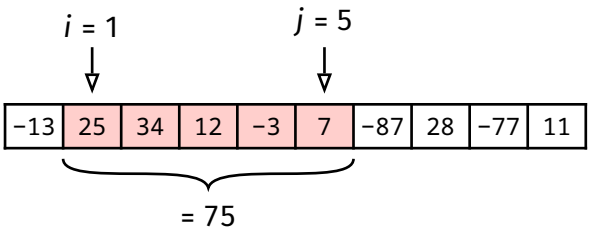
```
def inc(x, n, b):
  for i in range(n):
    if x[i] + 1 == b:
      x[i] = 0
    elif x[i] + 1 < b:
      x[i] += 1
      break
```

Im Fall $b = 2$ gilt für jedes bit $x + 1 = \neg x$. Die binäre Addition um 1 ist also noch einfacher:

```
def inc_bin(x, n):
  for i in range(n):
    x[i] = not x[i]
    if x[i] == 1:
      break
```

3.5 MaxTeilSum

Gesucht ist die Teilfolge, welche aufsummiert maximal ist.



Naiver Algorithmus (MTS1)

Am einfachsten wäre es, alle Möglichkeiten durchzuprobieren und das größte Ergebnis zu nehmen. Die möglichen Teilfolgen haben für jeden Startwert von $0 \leq i < n$ einen Endwert von $i \leq j < n$. Sie können also mit zwei for-loops durchlaufen werden.

```
for i in range(n):
  for j in range(i, n):
    # ...
```

Die Laufzeit für diesen Teil des Algorithmus ist $\Theta(n^2)$.

$$\begin{aligned}
\sum_{i=0}^{n-1}\sum_{j=i}^{n-1} 1 &= \sum_{i=0}^{n-1}(n-1-i) \\
&= n^2 - n - \sum_{i=0}^{n-1} i \quad \text{Gauß-Summe} \\
&= n^2 - n - \frac{n^2-n}{2} \\
&= \frac{1}{2}\cdot\left(n^2-n\right) \\
&= \Theta\left(n^2\right) \blacksquare
\end{aligned}$$

Für jede Teilfolge muss nur noch die Summe berechnet werden, sodass man die maximale Summe auswählen kann.

```
sum(array[i:j])
```

Die Laufzeit für die Berechnung der Summe ist $\Theta(n)$, weil eine Operation für jedes Element in `array[i:j]` ausgeführt wird, und dieses Array eine Länge von $\Theta(n)$ hat.

$$\begin{aligned}
S^{\text{AC}}_{\text{array}}(n) &= \sum_{i=1}^{n}\left(\frac{1}{2}\right)^i \\
&= \frac{n}{2} = \Theta(n)
\end{aligned}$$

Die Laufzeit für den naiven Algorithmus ist damit $\Theta\left(n^2 \cdot n\right) = \Theta\left(n^3\right)$.

Quadratischer Algorithmus (MTS2)

Dadurch, dass bereits in der inneren Schleife alle Elemente von i nach j angeschaut werden, ist eine nochmalige Iteration zur Summierung redundant.

```
for i in range(n):
  sum = 0
  for j in range(i, n):
    sum += array[j]
```

Damit wurde die Laufzeit auf $\Theta\left(n^2\right)$ reduziert.

Algorithmus von Kadane (MTS3)

Der Algorithmus von Kadane läuft eine Liste von links nach rechts durch, und hat deshalb eine lineare Laufzeit.

Σ	Liste									
-13	-13	25	34	12	-3	7	-87	28	-77	11
25	-13	25	34	12	-3	7	-87	28	-77	11
59	-13	25	34	12	-3	7	-87	28	-77	11
71	-13	25	34	12	-3	7	-87	28	-77	11
68	-13	25	34	12	-3	7	-87	28	-77	11
75	-13	25	34	12	-3	7	-87	28	-77	11
-12	-13	25	34	12	-3	7	-87	28	-77	11
28	-13	25	34	12	-3	7	-87	28	-77	11
-49	-13	25	34	12	-3	7	-87	28	-77	11
11	-13	25	34	12	-3	7	-87	28	-77	11

Für jedes Element x wird `aktSum` berechnet (erste Spalte). Das ist die maximale Teilsumme, welche an dieser Stelle endet. Beweis: Abschnitt 1.3

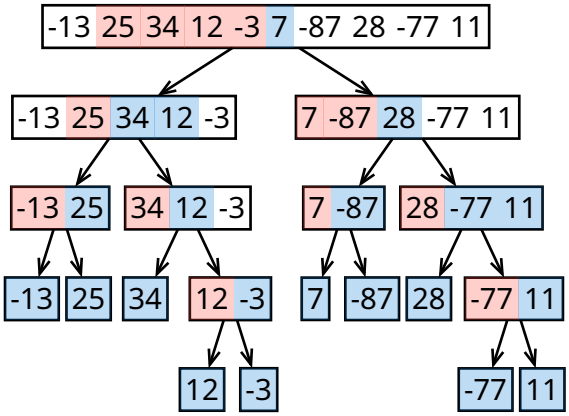
```
aktSum = max(x, aktSum + x)
```

Die maximale Teilsumme ist das Maximum aller dieser Teilsummen.

Divide and Conquer (MTS4)

Dieser rekursive Algorithmus teilt die Eingabe in jedem Schritt durch die Hälfte. Für jeden Knoten im Rekursionsbaum wird

die maximale Teilsumme, welche durch die Mitte geht, berechnet (farbig hinterlegt).



Das Ergebnis muss eine dieser Teilsummen sein, weil es entweder in der linken oder der rechten Hälfte liegt, oder durch die Mitte geht.

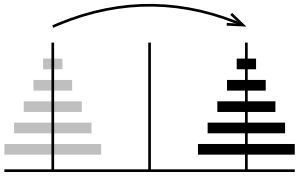
```
def mts4(nums):
  if nums.len() == 1:
    return nums.first()

  return max(
    mts4(left_half),
    mts4(right_half),
    center_mts(nums)
  )
```

Die Maximale Teilsumme, welche jeweils durch die Mitte eines Segments geht, wird durch eine Iteration nach links und nach rechts berechnet.

```
s = 0
m = -math.inf
for x in left.rev(): # same for right
  s += x
  m = max(m, s)
```

3.6 Türme von Hanoi

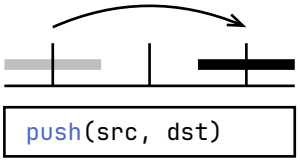
Es gibt drei Stäbe und n Scheiben mit unterschiedlichen Durchmessern. Zu Beginn sind alle Scheiben auf dem linken Stab. Ziel ist es, den gesamten Stapel Scheibenweise auf den rechten Stab zu verlegen. In jedem Schritt muss jede Scheibe auf einer größeren liegen. [3]



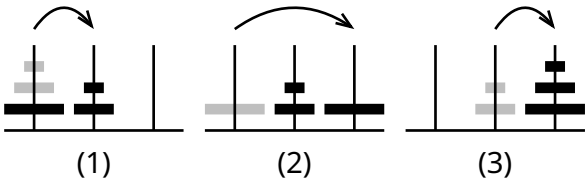
Die Hanoi-Funktion liefert die notwendigen Schritte, um n Scheiben von `src` auf `dst` zu verlegen.

```
(n, src, dst) -> ((from, to), ...)
```

Bei $n = 1$ muss die Scheibe lediglich auf den Zielstab gelegt werden.



Bei allen anderen n gibt es drei Schritte:



1. Verlege Scheiben $(1, 2, \ldots, n-1)$ von `src` auf `other`.

   ```
   hanoi(n-1, src, other)
   ```

2. Verlege Scheibe n von `src` auf `dst`.

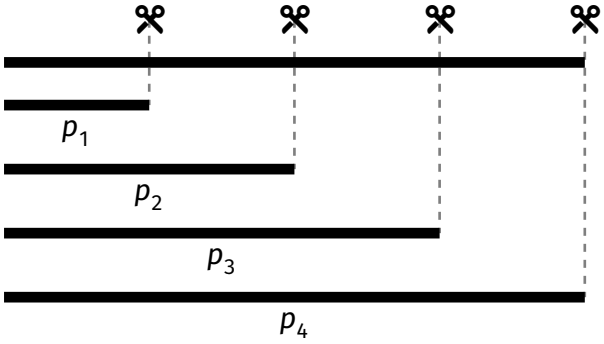
   ```
   push(src, dst)
   ```

3. Verlege Scheiben $(1, 2, \ldots, n-1)$ von `other` auf `dst`.

   ```
   hanoi(n-1, other, dst)
   ```

3.7 Rod Cutting Problem

Ziel ist es, einen Stab der Länge n in kleineren Stücken zu verkaufen, um den maximalen Gewinn zu erzielen, wobei jedem Schnittstück ein Preis zugeordnet ist [4].



Naiver Algorithmus

Man könnte alle möglichen Kombinationen durchprobieren und das Maximum nehmen. Es gibt $n-1$ Stellen, an denen der Stab geschnitten werden darf. Deshalb gibt es 2^{n-1} Möglichkeiten, den Stab zu zerstückeln.

$T(n) = \Theta(2^n)$

Tatsächlich gibt es weniger Kombinationen, weil dieser Ansatz die Reihenfolge der Stücke berücksichtigt. Um die genaue Anzahl zu berechnen, müssen wir die Partitionsfunktion anwenden [5].

4 Sortieralgorithmen

4.1 Insertion Sort

Der Algorithmus iteriert durch alle Elemente ab dem zweiten (rot markiert) und schiebt den Wert so lange nach vorne, bis keine kleineren Werte mehr vor ihm stehen.

```
for j in range(1, n):
  # a[j] nach vorne schieben
```

Zum Schluss von jeder Iteration ist die Liste bis zur jeweiligen Stelle sortiert (grün markiert).

Das „nach vorne schieben“ erfolgt in drei Schritten:

1. Merke dir den Wert des Schlüssels (graue Spalte).

   ```
   key = a[j]
   ```

2. Iteriere von rechts nach links durch alle Elemente links des Schlüssels (blau markiert) und kopiere das Element jeweils um eine Position nach hinten, wenn es größer ist als das nächste.

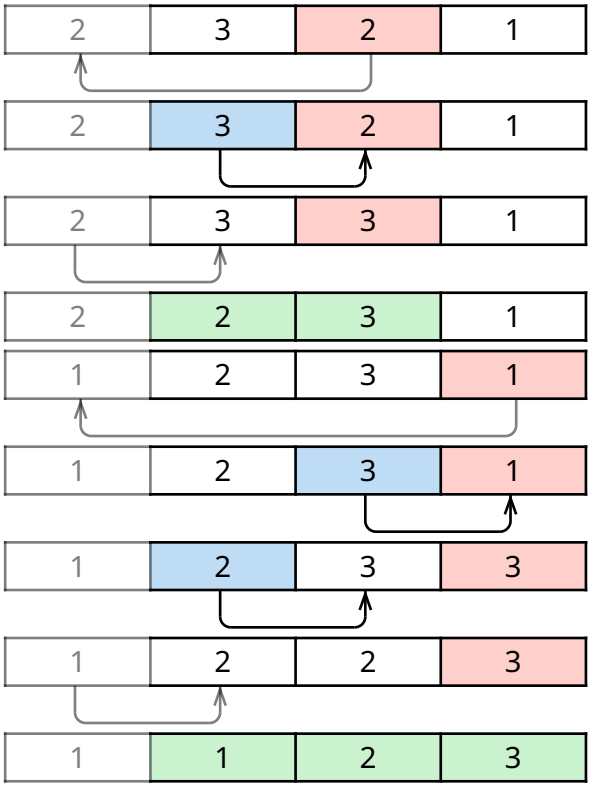
   ```
   i = j-1
   while (i ≥ 0 and a[i] > key):
     a[i+1] = a[i]
     i -= 1
   ```

3. Setze den Wert des Schlüssels ganz nach vorne.

   ```
   a[i+1] = key
   ```

Beispiel

Im folgenden Beispiel ist Insertion Sort für die Eingabeliste $(3, 2, 1)$ illustriert. Der Zähler (rot markiert) startet bei $i = 1$ und endet bei $i = n - 1 = 2$.



Längeres Beispiel: Abschnitt 2.2

Laufzeit

Für jedes Element `a[j]` werden zwischen 0 und j Verschiebeoperationen durchgeführt, um `a[j]` richtig in `a[:j+1]` einzusortieren.

$T^{WC}(n) = \Theta(n^2)$ $\qquad$ $T^{BC}(n) = \Theta(n)$

Im Schnitt benötigt es $j/2$ Verschiebeoperationen, damit ist die AC-Laufzeit

$$T^{AC}(n) = \sum_{j=1}^{n-1} \frac{j}{2} = \Theta(n^2)$$

Korrektheit

Behauptung: Nach jeden j-ten Schleifendurchlauf ist `a[:j+1]` sortiert.

Induktionsanfang ($j = 1$)

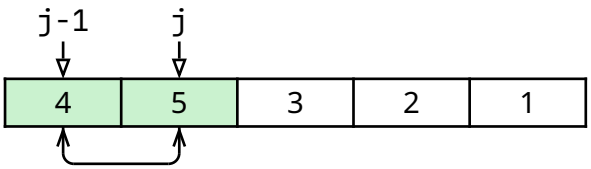
Ein Durchlauf der Schleife mit Zähler j sieht so aus:

```
key = a[j]
i = j-1
while i ⩾ 0 and a[i] > key:
  a[i+1] = a[i]
  i -= 1
a[i+1] = key
```

Setzen wir den festen Wert $j = 1$ in den Code ein, dann wird die while-Schleife höchstens einmal betreten. Vereinfacht kann man das als if-else-Block darstellen:

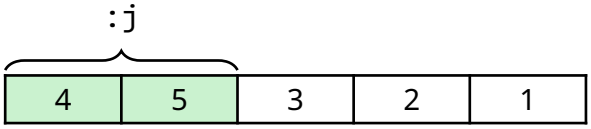
```
key = a[1]
if a[0] > a[1]:
  a[1] = a[0]
  a[0] = key
else:
  a[1] = key
```

Dieser Code vertauscht `a[0]` und `a[1]`, wenn `a[0]` größer ist. Das heißt, dass `a[0:2]` sortiert sind. ✓



Induktionsschritt ($j - 1 \longrightarrow j$)

Wir gehen davon aus, dass `a[:j]` nach Iteration $j - 1$ sortiert ist.



Wenn wir zeigen können, dass nach der nächsten Iteration j das Subarray `a[:j+1]` sortiert ist, ist der Induktionsschritt erfüllt.

Verschachtelte Induktion

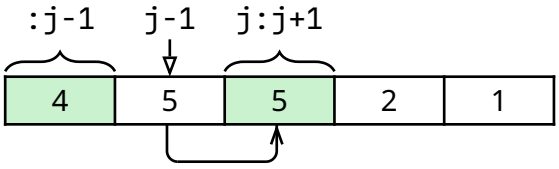
Die innere Schleife startet mit $i = j - 1$ und läuft ggf. nach unten bis $i = 0$. Behauptung: Nach jedem i-ten Schleifendurchlauf gilt sind die Elemente in `a[i+1:j+1]` an der richtigen Stelle.

Induktionsanfang ($i = j - 1$)

Im ersten Durchlauf mit $i = j - 1$ werden folgende Anweisungen ausgeführt:

```
if a[j-1] > a[j]:
  a[j] = a[j-1]
```

Weil wir annehmen, dass `a[:j]` sortiert ist, ist `a[j-1]` der größte Wert in `a[:j]`. Nach der Ausführung des obigen Codes ist `a[j]` der größte Wert aus `a[:j+1]`. Damit ist `a[j:j+1]` sortiert. `a[:j-1]` bleibt unberührt. ✓



Induktionsschritt ($i + 1 \longrightarrow i$)

Nach Induktionsvoraussetzung gilt nach dem Durchlauf mit $i + 1$:

`a[i+2:j+1]` ist korrekt

Wenn wir zeigen können, dass nach dem Durchlauf mit i gilt:

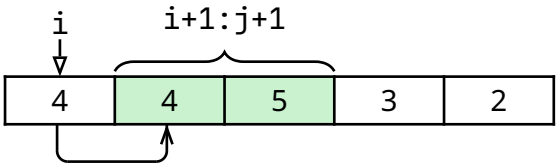
`a[i+1:j+1]` ist korrekt

Dann ist der Induktionsschritt erfüllt.

Falls $i + 1 = 0$ wird die while-Schleife verlassen, bevor sie für i ausgeführt werden kann. Wir beweisen also für dieses i nichts. ✓

Falls `a[i]` ≤ `key` wird die while-Schleife auch verlassen. ✓

Sonst wird `a[i]` an den Anfang der rechten sortierten Teilliste `a[i+1:j+1]` gesetzt.



Dort gehört dieser Wert auch hin, weil er durch die äußere IV (`a[:j]` ist sortiert) der größte noch nicht einsortierte Wert ist.

`a[:i]` ≤ `a[i]` ≤ `a[i+1:j+1]` ✓

Nach dieser inneren Schleifeninvariante sind `a[:i]` und `a[i+1:j+1]` nach Beendigung der While-Schleife mit irgend einem i sortiert. Ferner gilt folgendes:

`a[i-1]` ≤ `key` ≤ `a[i+1]`

Nachdem also der Schlüssel an Stelle `a[i]` setzt, ist `a[:j+1]` sortiert. ■

4.2 Bubble Sort

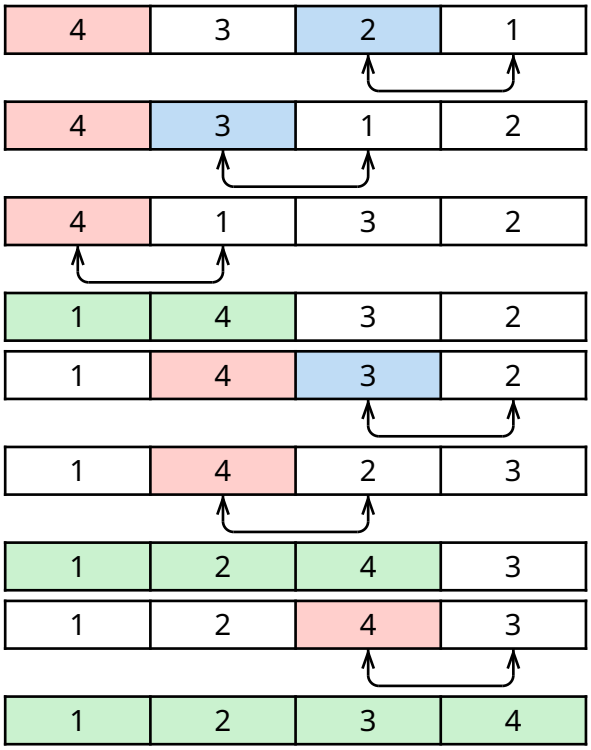
In jedem Schleifendurchlauf wird die rechte Teilliste `a[i:]` betrachtet. Von rechts nach links werden die Elemente `a[j]` und `a[j+1]` paarweise vertauscht, um den minimalen Wert nach vorne zu bringen.

```
for i in range(n):
  for j in reversed(range(i, n-1)):
    if a[j] > a[j+1]:
      a[j], a[j+1] = a[j+1], a[j]
```

Nach jedem Schleifendurchlauf ist die linke Teilliste bis zum Schlüssel sortiert (grün markiert).

Beispiel

Im Beispiel wird (4, 3, 2, 1) sortiert. Der Zähler (rot markiert) wandert von $i = 0$ nach $i = n - 2 = 2$.



Längeres Beispiel: Abschnitt 2.3

Laufzeit

Es gibt zwei verschachtelte for-Schleifen in Abhängigkeit von n.

$T^{WC}(n) = T^{BC}(n) = T^{AC}(n) = n^2$

Korrektheit

Nach jedem i-ten Schleifendurchlauf ist `a[:i+2]` sortiert.

Induktionsanfang ($i = 0$)

Verschachtelte Induktion

Nach jedem j-ten Schleifendurchlauf ist `min(a)` in `a[:j+1]` und `a[j]` ≤ `a[j+1]`

Induktionsanfang ($j = n - 2$)

Die letzten beiden Elemente `a[n-1]` und `a[n-2]` werden ggf. vertauscht, um sie in die richtige Reihenfolge zu bringen. ✓

Induktionsschritt ($j + 1 \longrightarrow j$)

Nach IV ist `min(a)` in `a[:j+2]`. Es werden `a[j]` und `a[j+1]` wieder ggf.

vertauscht, um sie in die richtige Reihenfolge zu bringen. ✓

Induktionsschritt (i - 1 ⟶ i)

Verschachtelte Induktion

Nach jedem *j*-ten Schleifendurchlauf ist `min(a[i:])` in `a[i:j+1]`.

Induktionsanfang (j = n - 2)

Siehe obige verschachtelte Induktion. ✓

Induktionsschritt (j + 1 ⟶ j)

Siehe obige verschachtelte Induktion. ✓

Induktionsschluss

Nach dem $n-1$-ten Schleifendurchlauf ist `a[:n+1]` sortiert. ■

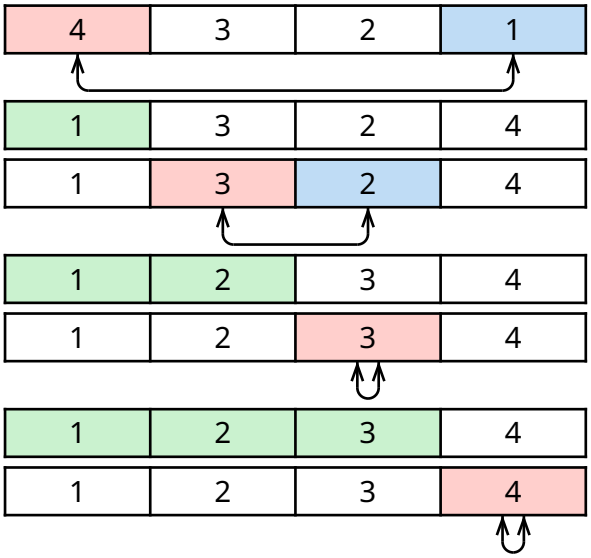
4.3 Selection Sort

In jedem Schleifendurchlauf ermitteln wir das Minimum in `a[i:]` und vertauschen es mit `a[i]`.

```
for i in range(n):
  x = a.index(min(a[i:]))
  a[i], a[x] = a[x], a[i]
```

Beispiel

Im Beispiel ist der Zähler rot markiert und das rechte Minimum blau.



Längeres Beispiel: Abschnitt 2.4

Laufzeit

$T^{WC}(n) = T^{BC}(n) = T^{AC}(n) = n^2$

Korrektheit

Nach jedem *i*-ten Schleifendurchlauf ist `a[:i+1]` sortiert.

Induktionsanfang (i = 0)

`a[0]` und `min(a)` werden vertauscht. (Verschachtelte Induktion für die Berechnung des Minimums wird weggelassen) ✓

Induktionsschritt (i - 1 ⟶ i)

`a[i]` und `min(a[i:])` werden vertauscht. `a[:i]` ist nach IV bereits sortiert. ✓

Induktionsschluss

Nach dem *n*-ten Schleifendurchlauf ist die gesamte Liste `a[:n+1]` sortiert. ■

4.4 Quicksort

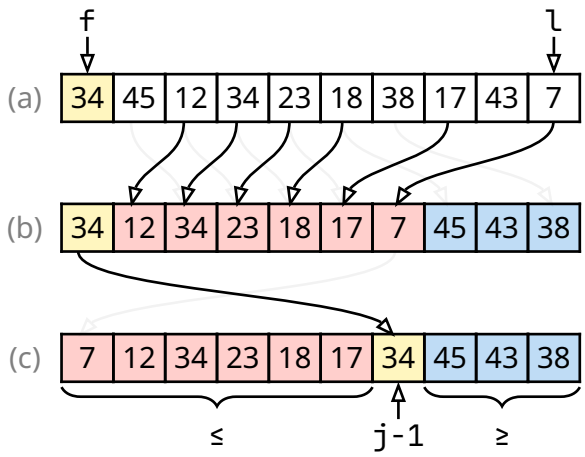
Um eine Liste `a[f:l+1]` zu sortieren, wird in jedem Rekursionsschritt wird ein beliebiges sogenanntes Pivot-Element ausgewählt. In unserer Implementation wählen wir

```
pivot = a[f]
```

Die Partitionsfunktion sortiert das Pivotelement ein. Die Sublisten links und rechts des Pivots werden anschließend rekursiv sortiert.

Partition

Die Partitionsfunktion vertauscht die Liste so, dass alle kleineren Elemente vor dem Pivot und alle größeren hinter ihm stehen.



Um von (a) auf (b) zu kommen, jede Zahl in `a[f+1:]` nach vorne vertauscht, falls sie kleiner als das Pivot ist.

```
j = f+1
for i in range(f+1, l+1):
  if a[i] ⩽ key:
    a[i], a[j] = a[j], a[i]
    j += 1
```

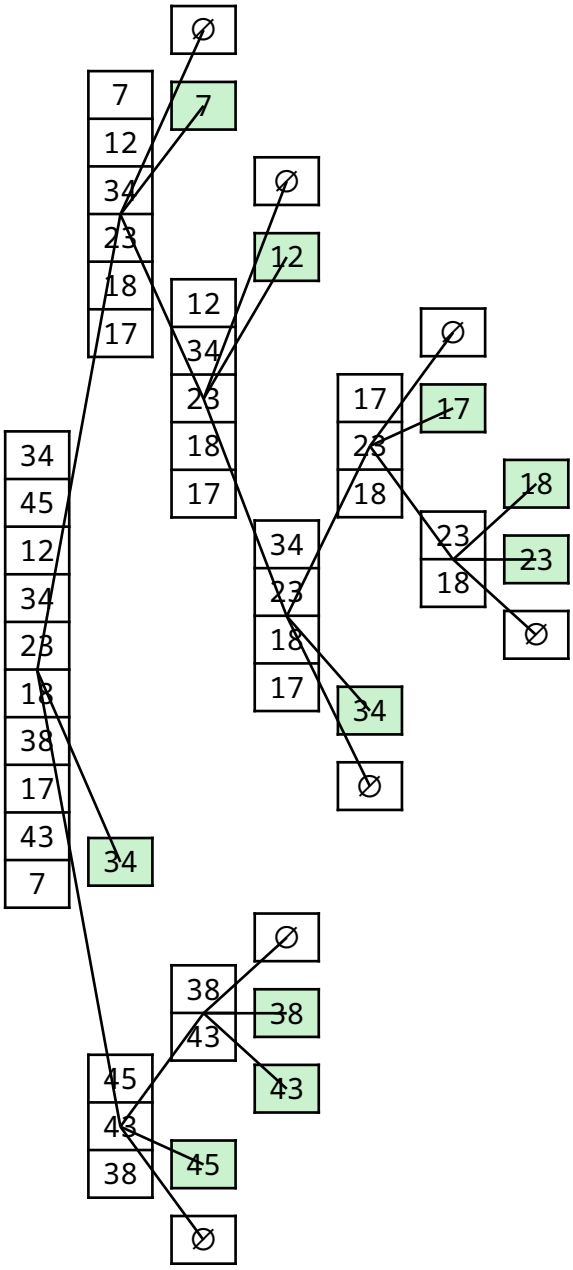
Jetzt ist `a[1:j]` kleiner, und `a[j:n+1]` größer als das Pivot. Wenn wir also das Pivot bei `a[f]` mit `a[j-1]` vertauschen, ist es an der richtigen Stelle (c).

```
a[f], a[j-1] = a[j-1], a[f]
```

Rekursion

Quicksort wird einmal für die Elemente links des Pivots und einmal für die rechts des Pivots aufgerufen. Falls es links oder rechts ein oder kein Element gibt, ist keine Aktion mehr erforderlich (base case).

```
def quicksort(a, f, l):
  if f ⩾ l:
    return # base case
  m = partition(a, f, l)
  quicksort(a, f, m-1)
  quicksort(a, m+1, l)
```

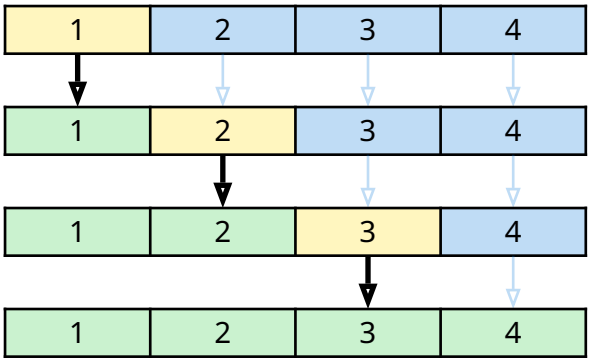


Laufzeit

Die Partition läuft die Eingabe einmal durch, hat also eine lineare Laufzeit $\Theta(n)$.

Worst Case

Wenn die Eingabe aufsteigend oder absteigend sortiert ist, wird das Problem in jedem Rekursionsschritt um nur 1 kleiner.



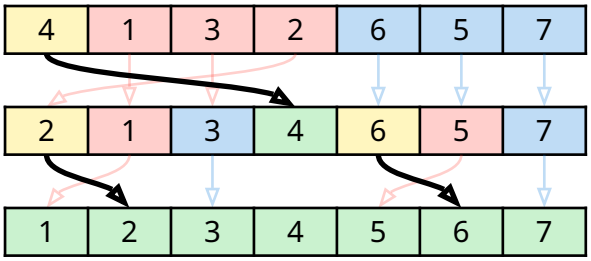
Ist sie aufsteigend sortiert, so wird nur der zweite rekursive Aufruf mit einer Eingabelänge von $n - 1$ aufgerufen.

$$T(n) = \begin{cases} \Theta(1) \text{ falls } n \leq 1 \\ T(n-1) + \Theta(n) \text{ sonst} \end{cases}$$

Die Rekursionsgleichung ist in $\Theta(n^2)$.

Best Case

Der Best-Case tritt auf, wenn das Pivot-Element in jedem Schritt die Eingabe halbiert.



In diesem Fall wird das Problem in jedem Schritt halbiert.

$$T(n) = \begin{cases} \Theta(1) \text{ falls } n \leq 1 \\ 2T\left(\frac{n}{2}\right) + \Theta(n) \text{ sonst} \end{cases}$$

Die Lösung dieser Rekursionsgleichung ist $\Theta(n \log n)$.

Average Case

Die Laufzeit hängt davon ab, an welcher Stelle m das Pivotelement landet. Ist immer $m = f$ oder $m = l$, haben wir den Worst Case. Ist immer $m = \lfloor (f + l)/2 \rfloor$, haben wir den Best Case.

Im allgemeinen gilt für beliebige $m \in \{f, \ldots, l\}$:

$$T_m(n) = \underbrace{T(m-f)}_{\text{links}} + \underbrace{T(l-m)}_{\text{rechts}} + \underbrace{\Theta(n)}_{\text{partition}}$$

Weil wir wissen, dass $l - f + 1 = n$, gilt für $i \in \{1, \ldots, n\}$:

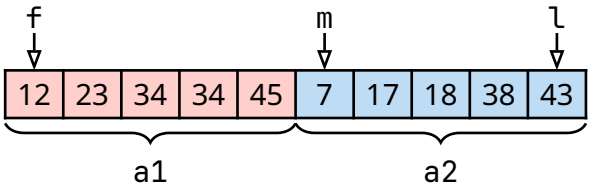
$$T_i(n) = T(i-1) + T(n-i) + \Theta(n)$$

4.5 Merge Sort

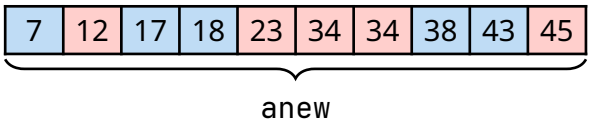
Genauso wie Quicksort spaltet Merge Sort die Eingabe in jedem Rekursionsschritt in zwei Sublisten.

Merge

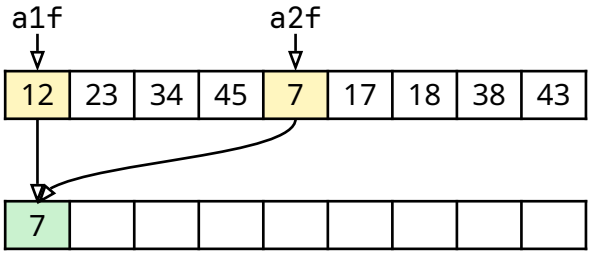
Die Merge-Prozedur nimmt als Eingabe zwei sortierte Listen `a1[f:m]` und `a2[m:l+1]`.



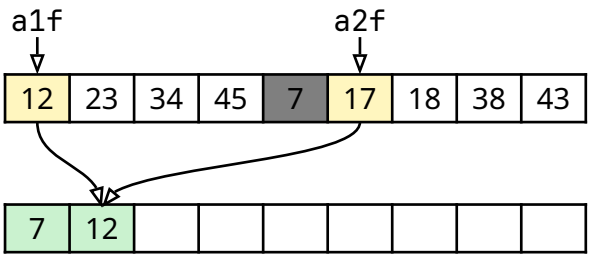
Die beiden Listen werden gemischt, um eine sortierte Liste `anew[f:l+1]` zu produzieren.



Um in Linearzeit zu mischen, nutzt quicksort, dass das Minimum zwei sortierter Listen `a1` und `a2` entweder `a1[0]` oder `a2[0]` sein muss.



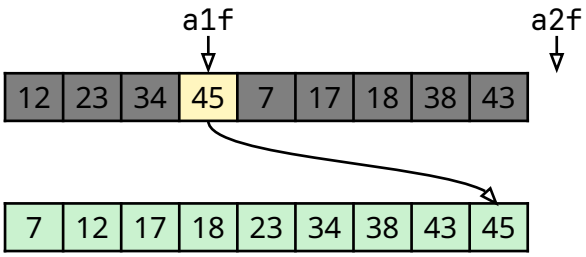
Streichen wir den kleineren Wert, so bleiben zwei sortierte Listen übrig, für die wiederum das gleiche gilt.



In Code kann das folgendermaßen implementiert werden:

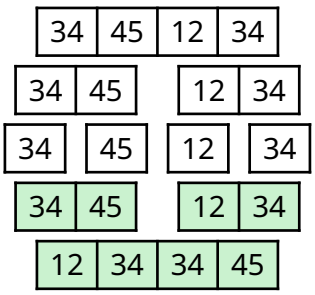
```
if a[a1f] ⩽ a[a2f]:
  anew[i] = a[a1f]
  a1f += 1
else:
  anew[i] = a[a2f]
  a2f += 1
```

Sobald die eine der beiden Listen durchgearbeitet wurde, wird der Rest der anderen Liste angehängt.



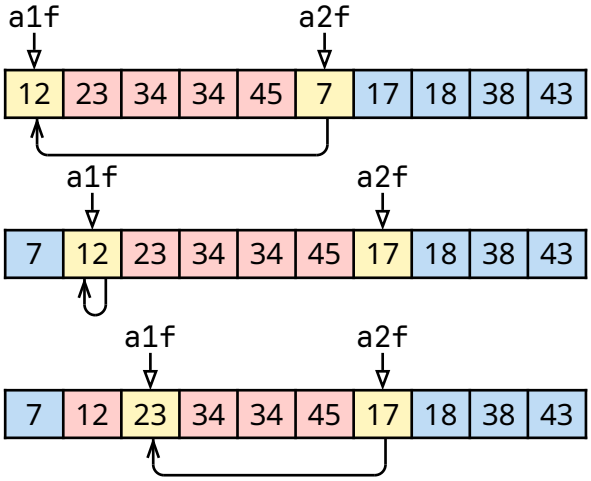
Rekursion

Die Eingabe wird rekursiv in 2 Teile gespalten, bis die Eingabe nur noch ein Element hat (base case). Die zwei Teile werden dann gemischt.



In Place

Um die Merge-Funktion ohne die hilfsliste anew zu implementieren, kann man Einträge der rechten Hälfte nach vorne verschieben, wenn sie kleiner sind. Weil in jedem Schritt entweder a[a2f] nach vorne gezogen wird oder a[a1f] stehen bleibt, wird a1f jedes mal inkrementiert, und a2f nur, wenn der Wert nach vorne gezogen wird.



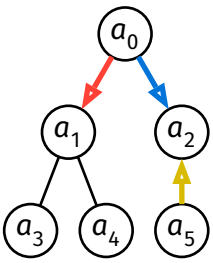
...

```
while a1f < a2f and a2f < l+1:
  if a[a2f] < a[a1f]:
    a.insert(a1f, a.pop(a2f))
    a2f += 1
  a1f += 1
```

4.6 Heap Sort

Linksvoller Baum

Ein Baum ist linksvoll, wenn alle Schichten bis auf die letzte voll besetzt sind, und die letzte von links nach rechts besetzt ist.



a_0	a_1	a_2	a_3	a_4	a_5

Linker NF	$2i + 1$
Rechter NF	$2i + 2$
Vorgänger	$\left\lfloor \frac{i-1}{2} \right\rfloor$

Heap

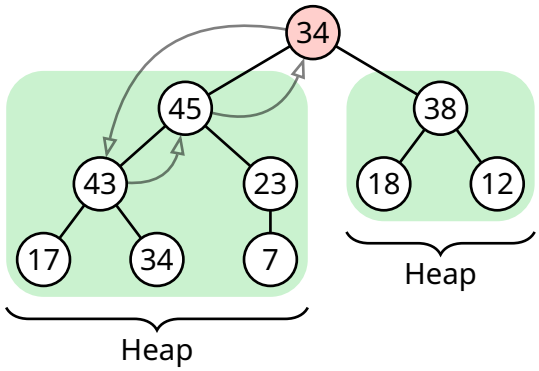
Ein Heap ist ein linksvoller Binärbaum, der für alle Knoten die Heap-Eigenschaft erfüllt:

Minheap: Vorgänger ≤ Nachfolger
Maxheap: Vorgänger ≥ Nachfolger

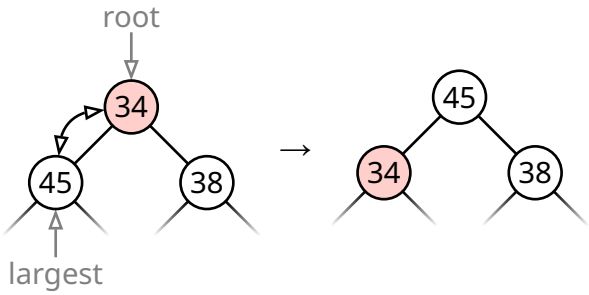
Heapify

Bevor wir ein beliebiges Array in einen Heap umwandeln, lösen wir ein einfacheres Problem: Ein Knoten a_i, dessen linker und rechter Teilbaum bereits Heaps sind, soll an die richtige Stelle vertauscht werden. Im folgenden implementieren wir Heapify für einen Maxheap. Betrachten wir als Beispiel

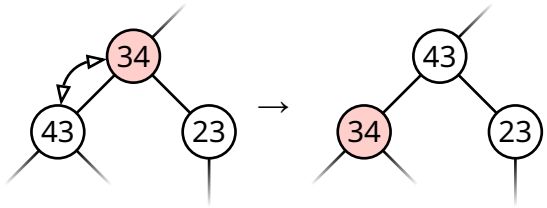
34	45	38	43	23	18	12	17	34	7



Zunächst vergleichen wir den Knoten mit seinen unmittelbaren Nachfolgern. Der maximale Wert muss nach oben, um die Heap-Eigenschaft zu erfüllen.



Durch den Tausch könnte die Heap-Eigenschaft jetzt im linken Teilbaum verletzt sein, also rufen wir Heapify rekursiv auf.



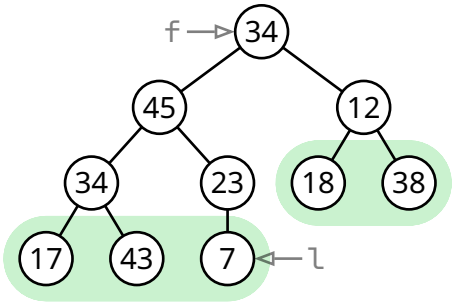
Die Rekursion ist dann beendet, wenn der Knoten größer als seine Nachfolger ist (oder keine Nachfolger hat), weil dann die Heap-Eigenschaft erfüllt ist.

Heapify hat eine Laufzeit von $O(\log n)$, weil die Wurzel im schlimmsten Fall nach ganz unten wandern muss.

BuildHeap

Wie bringen wir einen beliebigen linksvollen Baum in Max-Heap-Form? Alle Blätter sind bereits korrekte Heaps, weil sie keine Nachfolger haben. Beispiel:

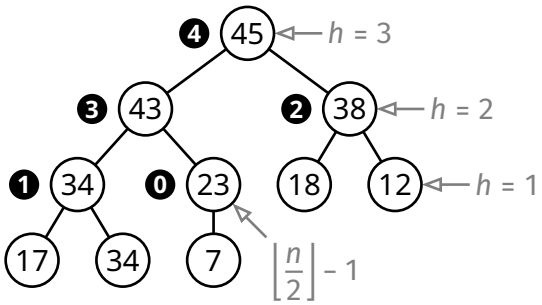
34	45	12	34	23	18	38	17	43	7



Wenn wir von ganz unten bis zur Wurzel für alle nicht-Blätter Heapify aufrufen, dann wird der gesamte Baum schrittweise zu einem Heap. Die nicht-Blätter sind in `a[:n//2]`, weil der Vorgänger des letzten Blattes an Stelle $n - 1$ ist:

$$\left\lfloor \frac{n-1-1}{2} \right\rfloor = \left\lfloor \frac{n}{2} \right\rfloor - 1$$

Wir iterieren von rechts nach links durch `a[:n//2]` und rufen für jeden Knoten Heapify auf.



Da wir für $n/2$ Knoten Heapify aufrufen, ist die Laufzeit von BuildHeap durch $O(n \log n)$ nach oben beschränkt. Tatsächlich ist BuildHeap noch schneller, weil die

meisten Knoten, für die Heapify aufgerufen wird, weit unten sind.

$$T^{WC}(n) = \sum_{h=0}^{\lfloor \log n \rfloor} \underbrace{\lceil n/2^{h+1} \rceil}_{\substack{\text{Anz. Knoten} \\ \text{auf Höhe h}}} \cdot \underbrace{O(h)}_{\text{Heapify}} = O(n)$$

HeapSort

HeapSort nutzt die Eigenschaft des Max-Heaps, dass der größte Wert in der Wurzel `a[f]` steht.

Zuerst bauen wir einen Heap in $O(n)$ Zeit auf. Dann vertauschen wir `a[0]` und `a[n-1]`, was dazu führt, dass der größte Wert an der letzten Stelle steht. Im nächsten Schritt vernachlässigen wir `a[n-1]` und wiederholen das gleiche für die Subliste `a[:n-1]`. Um diese zu einem Heap zu machen, müssen wir nur Heapify aufrufen.

```
build_heap(a)
for i in range(n-1, 1, -1):
  a[0], a[n-1] = a[n-1], a[0]
  heapify(n=n-1, root=i)
```

4.7 Counting Sort

Countsort ist nicht vergleichsbasiert, weshalb die untere Schranke $\Omega(n \log n)$ nicht gilt. Es gibt aber die Einschränkung, dass der Wertebereich der Eingabeliste a endlich ist.

$$a = (a_0, a_1, a_2, \ldots, a_{n-1}) \text{ mit } a_i \in \{0, 1, 2, \ldots, k\}$$

Desto größer k ist, desto mehr Speicher wird in Anspruch genommen. Wenn $k > n$, ist die Laufzeit nicht mehr linear.

$$S(n, k) = \Theta(n + k)$$
$$T(n, k) = \Theta(n + k)$$

Funktionsweise

Es wird eine k-Elementige Liste b aus Nullen angelegt. Diese soll an jeder Stelle j speichern, wie oft j in A vorkommt.

$$b = (b_0, b_1, b_2, \ldots, b_{k-1})$$
$$= (0, 0, 0, \ldots, 0)$$

Zum Beispiel speichert b_0 die Anzahl an Nullen in A.

Wir zählen die Zahlen in Linearzeit:

```
for i in range(n):
  b[a[i]] += 1
```

Zum Schluss müssen die Einträge von B zurück in A kopiert werden.

```
i = 0
for j in range(n):
  while b[i] == 0:
    i += 1
  a[j] = i
  b[i] -= 1
```

4.8 Bucket Sort

Genauso wie bei Counting Sort trifft die Untere Schranke vergleichsbasierter Sortieralgorithmen nicht auf Bucket Sort zu. Dass Bucket Sort in Linearzeit laufen kann, muss die Eingabe a gleichverteilt sein. In der Implementierung in CLRS wird auch gefordert, dass die Zahlen der Eingabe in $[0; 1[$ liegen.

$$a = \left(a_i \mid a_i \sim \mathcal{U}[-1; 1[\right)$$

Funktionsweise

Es wird ein n-Elementiges Hilfsfeld b angelegt, in dem jedes Element wiederum eine Liste mit dynamischer Größe ist. Diese Listen sind die „Buckets“, in denen später die Zahlen aus a landen, welche in den jeweiligen Bereich fallen.

b_0	b_1	b_2	...	b_{n-1}
$[0; \frac{1}{n}[$	$[\frac{1}{n}; \frac{2}{n}[$	$[\frac{2}{n}; \frac{3}{n}[$		$[\frac{n-1}{n}; 1[$

Die Buckets werden mit Insertion Sort seperat sortiert, und zuletzt konkateniert.

4.9 Map Sort

Map Sort ist eine Abwandlung von Bucket Sort, bei der das Hilfsfeld b keine Buckets enthält. Stattdessen bei einer Kollision der einzufügende Wert nach rechts oder links verschoben, sodass b nach jeder Iteration sortiert ist.

Kollisionshandling

Das Hilfsfeld b ist vor jedem Durchlauf sortiert. Ist das Zielfeld bereits belegt, wird a_i mit einem modifizierten Insertion Sort in b einsortiert: Der Schlüssel wird als a_i initialisiert, und wandert von t aus nach rechts bzw. links.

Wenn t an ein Ende der Liste gelangt, bevor x einsortiert werden konnte, wird die Richtung geändert und ggf. bis ans andere Ende sortiert.

Erweiterung des Hilfsfeldes

Die Größe des Hilfsfeldes ist mindestens n, und kann um einen konstanten Faktor $c > 1$ vergrößert werden, um Kollisionen zu minimieren.

Erweiterung der erlaubten Eingabe

Map Sort erlaubt Zahlen in einem beliebigen Bereicht $[\mathrm{lo}; \mathrm{hi}]$, indem die jeweiligen Einträge a_i in den Bereich $[0, 1]$ skaliert werden.

$$\mathrm{normalize}(a_i) = \frac{a_i - \mathrm{lo}}{\mathrm{hi} - \mathrm{lo}}$$

Der Zielindex t ist dann

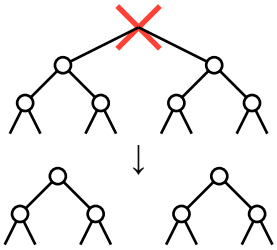
$$f(a_i) = \underbrace{\mathrm{normalize}(a_i)}_{\in[0;1]} \cdot \underbrace{(n \cdot c - 1)}_{\mathrm{len}(b)-1}$$

5 Suchbäume

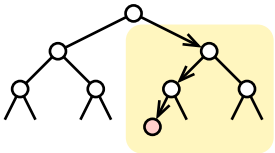
5.1 Binärer Suchbaum

Löschen (iterativ)

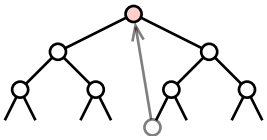
Löschen wir einen Knoten z mit zwei Nachfolgern, so bleiben zwei Teilbäume übrig, welche die BST-Eigenschaft erfüllen.



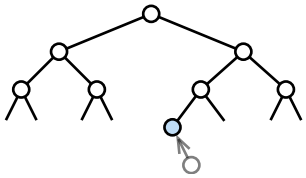
Jeder Knoten im rechten Teilbaum ist größer als jeder Knoten im Linken. Daraus folgt, dass die Wurzel des linken Teilbaums der linke Nachfolger aller dieser Knoten sein darf. Es gibt aber nur einen Knoten innerhalb des rechten Teilbaums, der die Wurzel des rechten Teilbaums als rechten Nachfolger übernehmen darf: das Minimum. (Umgekehrt gilt das gleiche für das *Maximum* des *linken* Teilbaums.)



Das Minimum des rechten Teilbaums ist der *Inorder-Nachfolger* des zu löschenden Knoten. Wir wählen diesen als neue Wurzel.



Falls der Inorder-Nachfolger einen rechten Nachfolger hatte, wird dieser nach oben geschoben.



Löschen (rekursiv)

Wir kopieren den Wert des Inorder-Nachfolgers in die Wurzel, und löschen dann rekursiv den Inorder-Nachfolger. Der Base-Case ist erreicht, wenn der zu löschende Knoten weniger als zwei Nachfolger hat. Das ist schon nach einem Selbstaufruf,

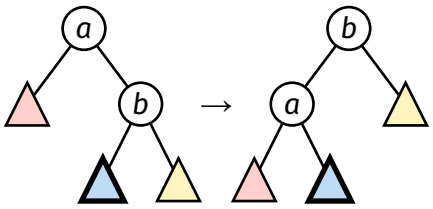
weil der Inorder-Nachfolger keinen linken Nachfolger haben kann.

5.2 AVL-Baum

Ein AVL-Baum ist ein selbstbalancierender BST. In einem AVL-Baum gilt für jeden Knoten, dass die Höhe des linken und rechten Teilbaums sich um maximal 1 unterscheiden [6], [7].

$$\mathrm{BF}(x) = h(B_R) - h(B_L), \quad |\mathrm{BF}(x)| \leq 1$$

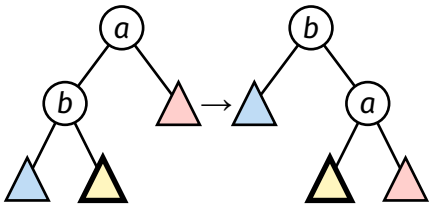
Linksrotation



Die Höhen verändern sich möglicherweise.

Knoten	Davor	Danach
a	$\max(\triangle, b) + 1$	$\max(\triangle, \triangle) + 1$
b	$\max(\triangle, \triangle) + 1$	$\max(a, \triangle) + 1$

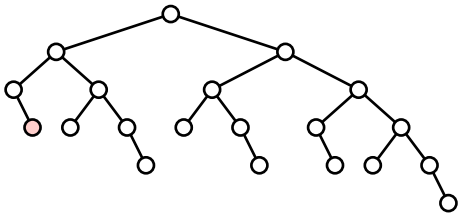
Rechtsrotation



Knoten	Davor	Danach
a	$\max(b, \triangle) + 1$	$\max(\triangle, \triangle) + 1$
b	$\max(\triangle, \triangle) + 1$	$\max(\triangle, a) + 1$

Löschen

Wenn wir die BST-Implementierung zum Löschen verwenden, erfüllt der resultierende Baum möglicherweise nicht mehr die AVL-Eigenschaft [8]. Um sie wieder herzustellen, laufen wir vom Vorgänger des Vorgänger des gelöschten Knoten bis zur Wurzel und rebalancieren jegliche Knoten, die unbalanciert sind.



5.3 RS-Baum

Ein Rot-Schwarz Baum (RBT) ist ein selbstbalancierender BST. Der kürzestmögliche Pfad in einem RBT besteht nur aus schwarzen Knoten, und der längstmögliche aus abwechselnd roten und schwarzen Knoten. Daher ist die höhe maximal

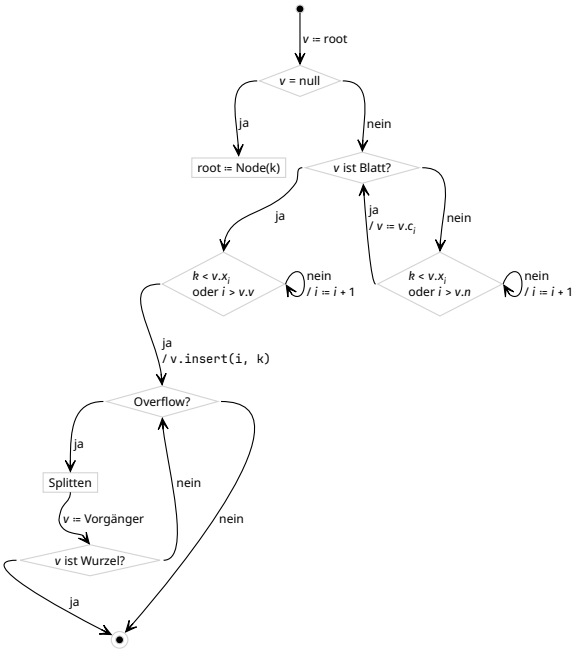
$$h \leq 2 \log_2(n)$$

5.4 B-Baum

Dadurch, dass jeder Knoten mindestens $\lceil t/2 \rceil \geq 2$ Nachfolger hat, ist die Höhe garantiert logarithmisch.

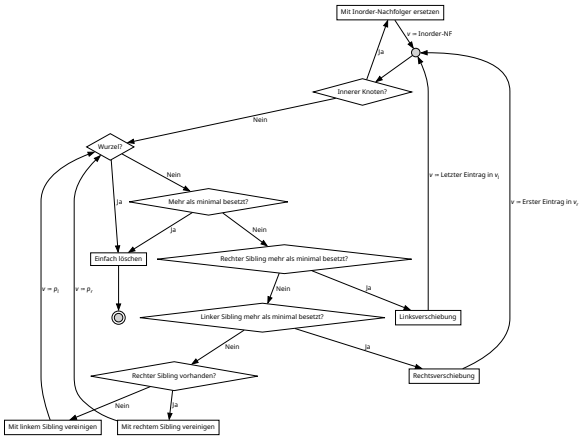
$$h \leq \log_{\lceil t/2 \rceil}(n)$$

Einfügen



Löschen

Dieses Flowchart beschreibt das Löschen eines Knotens und die Wiederherstellung der B-Baum-Eigenschaften [9].



Unterschiede zu CLRS

- In der Vorlesung hat ein t-ärer B-Baum maximal t Nachfolger, in CLRS maximal $2t$.
- In der Implementierung von CLRS werden die B-Baum Eigenschaften beim Einfügen und Löschen nicht gebrochen und wiederhergestellt, sondern präventiv vorgebeugt.

6 Datenstrukturen

6.1 Hashtabelle

Eine Hashtabelle kann verwendet werden, um ein Set mit konstanten Zugriffzeiten zu implementieren. Ein Element in dieser Menge ist ein *Schlüssel*. Wenn man jedem Schlüssel einen Wert zuweist, dann ist es ein Dictionary [10].

Unter der Haube einer Hashtabelle befindet sich ein statisches Array. Wenn man auf einen Eintrag mit einem Schlüssel s zugreifen möchte, wird der entsprechende Index mit einer *Hashfunktion* $h(s)$ berechnet.

Um es einfach zu halten, implementieren wir ein Set, dessen Schlüssel nichtnegative ganze Zahlen sind.

Hashfunktion

Ziel ist es, die Schlüssel möglichst gleichverteilt auf die Hashtabelle zu verteilen, um Kollisionen zu vermeiden. auf einen

Divisionsmethode

Eine einfache Modulooperation stellt sicher, dass ein beliebiger Schlüssel $s \in \mathbb{N}_0$ einem Index zwischen 0 und $m - 1$ zugeordnet wird.

$$h(s) = s \bmod m$$

Wenn mehrere Schlüssel sich lediglich um ein Vielfaches von a unterscheiden, dann müssen sie sich $\frac{m}{\gcd(a,m)}$ Zellen teilen [11]. Daher wäre es optimal, wenn $\gcd(a, m) = 1$. Das ist erfüllt, wenn a und m teilerfremd sind. Wenn wir für m eine Primzahl wählen, dann ist es mit allen $a < m$ teilerfremd.

Multiplikationsmethode

Wähle einen Faktor $x \in]0, 1]$. Der Bruchteil bestimmt die Position in der Hashtabelle.

$$h_x(s) = \lfloor ((x \cdot s) \bmod_c 1) \cdot m \rfloor$$

Kollisionsbehandlung

Siehe Abschnitt 9.4. Eine schlechte Hashfunktion kann in eine Unendlichschleife geraten.

Lineares Probieren

$$h(s, i) = \acute{h}(s) + i$$

Quadratisches Probieren

$$h(s, i) = \acute{h}(s) + c_1 i + c_2 i^2$$

Vermeidet Primary Clustering [12].

Doppeltes Hashing

$$h(s, i) = \acute{h}(s) + i h_2(s)$$

6.2 Skip-Liste

Eine Skip-Liste ist eine linear verkettete Liste, dessen Knoten nicht nur einen, sondern beliebig viele Nachfolger haben können. Die Höhe eines Knotens ist der Index des letzten Nachfolgers.

```
@dataclass
class Node:
    key: Any
```

```
    height: int
    next: list[Node]
```

Die Höhe wird unabhängig zufällig ausgewählt. Jede Höhe ist halb so Wahrscheinlich wie die vorherige, sodass die Skipliste eine erwartete Höhe von $\log n$ hat [13].

```
height = np.random.geometric(p=.5)
```

Bei einer Suche werden die Ebenen von oben nach unten iteriert, und für jede Ebene wird so lange nach rechts gesprungen, bis der nächste Wert zu groß wäre [14].

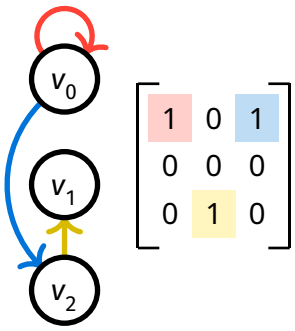
```
current = head
for i in range(height, -1, -1):
  while current.next[i] and
        current.next[i].key ≤ key:
    current = current.next[i]
```

7 Graphen

7.1 Speicherung

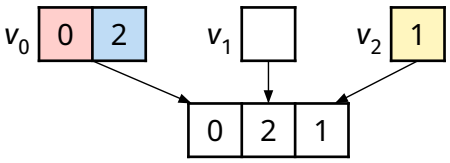
Adjazenzmatrix

Der Knoten v_i in Zeile i zeigt auf die Knoten v_j in Spalte j, wenn $a_{ij} = 1$ ist.

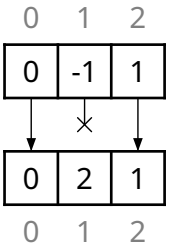


Adjazenzliste (dicht)

Wir speichern für jeden Knoten seine ausgehenden Kanten in einer flachen Kantenliste.



Um den ausgehenden Kanten ihre Knoten zuzuordnen, erstellen wir eine Zuordnungsliste, welche an Index i für den Knoten v_i speichert, welches Segment diesem Knoten gehört.



Das Segment wird am Index der ersten Kante identifiziert. hat der Knoten keine ausgehende Kanten, so wird ein Sentinel (-1) verwendet.

Adjazenzliste (gestreut)

Verkettete Liste mit zwei next-Pointer:

- Nächster Knoten (nach unten)
- Nächste Kante (nach rechts)

Adjazenzliste (hybrid)

Liste, dessen Eintrag an Index i auf eine Verkettete Liste zeigt, welche die ausgehenden Kanten von v_i hat.

7.2 Durchlaufen

Noah Jutz

CC 0

Algorithmen und Datenstrukturen Übungen

1 Algorithmen 25
1.1 ggT / kgV 25
1.2 Eratosthenes 25
1.3 Matrix 25
1.4 Matrix Laufzeit 25
2 Komplexität 26
2.1 Registermaschine 26
2.2 O-Notation 26
2.3 Formel von Binet 28
2.4 O-Notation 29
3 Rekursion 30
3.1 MaxTeilSum-2d 30
3.2 Iteration und Substitution 31
3.3 Master-Methode 32
3.4 Ackermannfunktion 33
4 Sortieralgorithmen 34
4.1 Sortieralgorithmen modifizieren . 34
4.2 Zeitmessungen 35
4.3 Minkowski-Summe 35
4.4 Iterativ/Rekursiv 36
5 Heaps 36
5.1 Mergesort und Heapsort 36
5.2 BuildHeap Laufzeit Aussagen 39
5.3 Strassen 40
5.4 Strassen Implementierung 42
6 Datenstrukturen 43
6.1 Sortieren in Linearzeit 43
6.2 Verkettete Liste 43
6.3 Ringliste 44
6.4 Rucksackproblem 44
7 BST und AVL-Bäume 45
7.1 BST 45
7.2 BST Eindeutigkeit 46
7.3 AVL Einfügen und Löschen 46
7.4 Minimaler AVL-Baum 46
8 RS- und B-Bäume 47
8.1 RS-Baum 47
8.2 B-Baum Einfügen 47
8.3 B-Baum Löschen 47
8.4 B-Bäume konstruieren 47
9 Hashing 47
9.1 Prioritätswarteschlange 47
9.2 Multiplikationsmethode 48
9.3 Erwartungswert Kollisionen 48
9.4 Kollisionsauflösung 48
10 Textsuche 48
10.1 Skip-Liste 48
10.2 Suche ausführen 49
11 Graphen 49
11.1 Adjazenzmatrix 49

1 Algorithmen

1.1 ggT / kgV

1.2 Eratosthenes

1.3 Matrix

Eine $(m \times n)$ Matrix hat m Zeilen und n Spalten.

$$A = \left(a_{ij}\right)_{\substack{1 \le i \le m \\ 1 \le i \le n}}$$

Addition

Zwei Matrizen A und B müssen die gleiche Anzahl an Zeilen und Spalten haben, um addiert werden zu können. Das Ergebnis C hat auch die gleiche Anzahl an Zeilen und Spalten. Es wird durch komponentenweise Addition berechnet:

$$c_{ij} = a_{ij} + b_{ij}$$

In Python lässt sich das folgendermaßen implementieren:

```
for i in range(m):
  for j in range(n):
    c[i][j] = a[i][j] + b[i][j]
```

Multiplikation

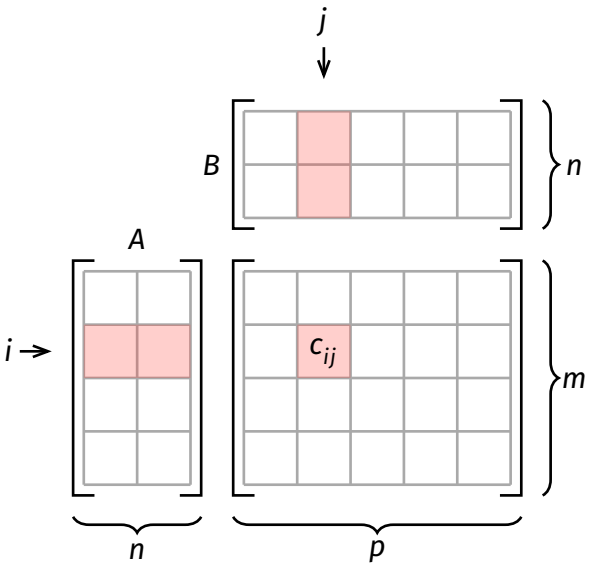
Die Spalten der ersten Matrix muss gleich der Zeilen der Zweiten sein.

$$A.n = B.m$$

Das Ergebnis hat die Zeilen der ersten Matrix und die Spalten der Zweiten.

$$\left(c_{ij}\right)_{\substack{1 \le i \le A.m \\ 1 \le j \le B.n}}$$

Jeder Ergebniseintrag c_{ij} errechnet sich aus der i-ten Zeile in A und der j-ten Spalte in B.



Konkret wird für jede Spalte in A und jede Zeile in B das Produkt von a_{ik} und b_{kj} berechnet. Alle Produkte werden anschließend aufsummiert.

$$c_{ij} = \sum_{k=1}^{n} a_{ik} \cdot b_{kj}$$

In Python:

```
for i in range(m):
  for j in range(p):
    for k in range(n):
      c[i][j] += a[i][k] * b[k][j]
```

1.4 Matrix Laufzeit

Die Addition zwei quadratischer Matrizen hat eine Laufzeit von

$$T_+ = m \cdot n = n^2$$

weil es so viele Einträge gibt. Die Multiplikation hat eine Laufzeit von

$$T_\times = m \cdot p \cdot n = n^3$$

weil für jeden Eintrag zusätzlich n Multiplikationen und Additionen ausgeführt werden.

Zeitkomplexität

Mit $n = 1000$ benötigt mein Laptop für die Matrixmultiplikation $t = 175\text{s}$. Daraus ergibt sich die Frequenz

$$
\begin{aligned}
f &= \frac{T(n)}{t} \\
&= \frac{1000^3}{175} \\
&\approx 5.7\ \text{MHz}
\end{aligned}
$$

Mit meiner Prozessorleistung kann ich in einer gegebenen Dauer t zwei Matrizen der Größe

$$
\begin{aligned}
n(t) &= T^{-1}(t \cdot f) \\
&= \sqrt[3]{t \cdot 5.7\ \text{MHz}}
\end{aligned}
$$

t	n (×)	n (+)
1 min	699	$1.7 \cdot 10^4$
2 min	881	$2.5 \cdot 10^4$
5 min	1195	$3.9 \cdot 10^4$
10 min	1506	$5.6 \cdot 10^4$

Platzkomplexität

Mit $n = 2000$ nimmt eine Matrix 92 MB ein. Die Anzahl speicherbarer Elemente x pro Byte ist also

$$
\begin{aligned}
x &= \frac{P(n)}{s} \\
&= \frac{2000^2}{92 \cdot 10^6} \\
&\approx 0.04
\end{aligned}
$$

Hochgerechnet benötigt eine $10^4 \times 10^4$ Matrix

$$
\begin{aligned}
s &= \frac{P(n)}{x} \\
&= \frac{10^8}{0.04} \\
&= 2.5\ \text{GB}
\end{aligned}
$$

Bei der Addition ist also die Platzkomplexität der beschränkende Faktor.

2 Komplexität

2.1 Registermaschine

Die Formel lässt sich algorithmisch folgendermaßen definieren:

$$\sum_{i=1}^{n} i^3 \iff$$

```
result = 0
for i in range(n):
    result += i ** 3
```

Ein For-Loop kann mit Jumps realisiert werden, und eine Potenz lässt sich als Aneinanderreihung mehrerer MUL-Befehle umschreiben.

LDK 1	$f(1) \coloneqq 1$
STA 1	
INP 2	$i \coloneqq$ input
LDA 2	$f(0) \coloneqq i^3$
MUL 2	
MUL 2	
ADD 3	$r \coloneqq r + f(0)$
STA 3	
LDA 2	$i \coloneqq i - 1$
SUB 1	
STA 2	
JGZ 4	for $i > 0$
OUT 3	Ausgabe
HLT 99	

Der Zustand einer Registermaschine setzt sich aus Instruction-Pointer und Speicherzellen zusammen. In jeder Iteration führt er die aktuelle Anweisung aus und manipuliert dabei seinen Zustand. In Python kann man das als Iterable umsetzen.

```
cmd, arg = instructions[ip]
if cmd == "hlt":
    raise StopIteration

getattr(commands, cmd)(arg)
return [ip, mem]
```

2.2 O-Notation

Aussage 1

$17 + 22 + 45 = O(1)$

Beweis

$$O(f(n)) = \left\{ g(n) \;\middle|\; \begin{array}{l} \exists c > 0, n_0 \in \mathbb{N} : \\ \forall n \geq n_0 : g(n) \leq c \cdot f(n) \end{array} \right.$$

$$\begin{aligned} O(1) &= \left\{ g(n) \;\middle|\; \begin{array}{l} \exists c > 0, n_0 \in \mathbb{N} : \\ \forall n \geq n_0 : g(n) \leq c \end{array} \right. \\ &= \{g(n) \mid \exists c > 0 : g(n) \leq c\} \\ &= \mathbb{R}^+ \\ &\ni 17 + 22 + 45 \;\blacksquare \end{aligned}$$

Aussage 2

$5n^3 + 12n^2 + 3n + 5 = \Omega\left(n^3\right)$

Beweis

$$\begin{aligned} \Omega(f(n)) &= \left\{ g(n) \;\middle|\; \begin{array}{l} \exists c > 0, n_0 \in \mathbb{N} : \\ \forall n \geq n_0 : g(n) \geq c \cdot f(n) \end{array} \right. \\ &\supset \{g(n) \mid g(n) \geq f(n)\} \\ \Omega\left(n^3\right) &\supset \left\{g(n) \mid g(n) \geq n^3\right\} \\ &\ni 5n^3 + 12n^2 + 3n + 5 \end{aligned}$$

Weil $n \nless 0$. ■

Aussage 3

$2^{n+1} = O(2^n)$

Beweis

$$\begin{aligned} & 2^{n+1} = \mathbf{2} \cdot 2^n \\ \Leftrightarrow\; & g(n) = 2 \cdot f(n) \\ \Leftrightarrow\; & g(n) = c \cdot f(n) \\ \Rightarrow\; & g(n) \in \Theta(f(n)) \\ \Rightarrow\; & g(n) \in O(f(n)) \;\blacksquare \end{aligned}$$

Aussage 4

$2^{2n} = O(2^n)$

Gegenbeweis

Reductio ad absurdum

$$\begin{aligned} 2^{2n} \in O(2^n) &\Rightarrow f(n) = 2^n \\ &\Rightarrow g(n) = 2^{2n} \end{aligned}$$

$$\begin{aligned} g(n) &\leq c \cdot f(n) \\ 2^{2n} &\leq c \cdot 2^n \\ (2^n)^2 &\leq c \cdot 2^n \\ 2^n &\leq c \;\lightning \end{aligned}$$

Es gibt kein $c > 0$, welches für alle $n > n_0$ die Funktion 2^n nach oben beschränkt. Daher ist die Annahme falsch. ■

Aussage 5

$\log(n!) = \Theta(n \log n)$

Beweis

Wir verwenden die Stirlingformel.

$$\begin{aligned} \log(n!) &\approx \log\left(\sqrt{2\pi n} \cdot \left(\frac{n}{e}\right)^n\right) \\ &= \log\left(\sqrt{2\pi n}\right) + \log\left(\left(\frac{n}{e}\right)^n\right) \\ &= \log\left((2\pi n)^{\frac{1}{2}}\right) + \log\left(\left(\frac{n}{e}\right)^n\right) \\ &= \frac{1}{2}\log(2\pi n) + n\log\left(\frac{n}{e}\right) \\ &= \frac{1}{2}\log(2\pi) + \frac{1}{2}\log(n) \\ &\quad + n\log(n) - n\log(e) \end{aligned}$$

Der dominierende Term ist $n \log n$. Eine Funktion $g(n)$ ist immer durch ihren dominierenden Term $f(n)$ nach oben und nach unten beschränkt, weil alle untergeordneten Terme durch ein passendes c ausgeglichen werden können. ■

Aussage 6

$2^n = O(n!)$

Beweis

$$\begin{aligned} \lim_{n\to\infty} \frac{2^n}{n!} &= \lim_{n\to\infty} \frac{2^n}{\sqrt{2\pi n} \cdot \left(\frac{n}{e}\right)^n} \\ &= \lim_{n\to\infty} \frac{2^n e^n}{\sqrt{2\pi n} \cdot n^n} \\ &= \lim_{n\to\infty} \frac{1}{\sqrt{2\pi}} \cdot \left(\frac{2e}{\sqrt{n} \cdot n}\right)^n \\ &= \lim_{n\to\infty} \frac{1}{\sqrt{2\pi}} \cdot \left(\lim_{n\to\infty} \frac{2e}{\sqrt{n} \cdot n}\right)^n \\ &= \frac{1}{\sqrt{2\pi}} \cdot (0)^n \\ &= 0 \\ &\leq c \;\blacksquare \end{aligned}$$

Aussage 7

$n! = O(n^n)$

Beweis

$$n! = \prod_{i=1}^{n} i \leq \prod_{i=1}^{n} n = n^n \quad \blacksquare$$

Aussage 8

$6^{-5} n^{1.25} = \Theta\left(\sqrt{n}\right)$

Gegenbeweis

Wir vernachlässigen den konstanten Faktor 6^{-5} und konzentrieren uns auf die obere Schranke.

$n^{1.25} \in O\left(n^{0.5}\right)$

Dafür ist notwendig, dass $n^{0.5}$ dominiert. Das ist nicht der Fall, damit ist die Annahme falsch.

$$\lim_{n\to\infty} \frac{n^{1.25}}{n^{0.5}} = \left(\lim_{n\to\infty} n\right)^{1.25-0.5} = \infty \quad \blacksquare$$

2.3 Formel von Binet

Die Glieder der Fibonacci-Folge f_n können durch die Formel von Binet berechnet werden.

Satz

$$\forall n \in \mathbb{N}_0 : f_n = \frac{\varphi^n - \psi^n}{\varphi - \psi}$$

mit $\varphi = \dfrac{1+\sqrt{5}}{2}$ und $\psi = \dfrac{1-\sqrt{5}}{2}$

Beweis

Anfang (n = 0, n = 1)

Wir Zeigen im Induktionsanfang, dass die Aussage für f_0 und f_1 gilt, sodass wir im Induktionsschritt für alle $f_2, f_3, \ldots$ auf die beiden Vorgänger zurückgreifen können.

$$f_0 = 0 = \frac{\varphi^0 - \psi^0}{\varphi - \psi} \checkmark$$

$$f_1 = 1 = \frac{\varphi - \psi}{\varphi - \psi} \checkmark$$

Schritt (n - 2, n - 1 ⟶ n)

Wir nehmen an, dass die Aussage für die Vorgänger f_{n-2} und f_{n-1} gilt. Wenn die Aussage auch für f_n gilt, dann ist bewiesen, dass sie für alle $n \in \mathbb{N}_0$ gilt.

$f_n = f_{n-1} + f_{n-2}$

Per Induktionsannahme dürfen wir einsetzen:

$$f_n = \frac{\varphi^{n-1} - \psi^{n-1}}{\varphi - \psi} + \frac{\varphi^{n-2} - \psi^{n-2}}{\varphi - \psi}$$

Umformen:

$$= \frac{\left(\varphi^{n-1} + \varphi^{n-2}\right) - \left(\psi^{n-1} + \psi^{n-2}\right)}{\varphi - \psi}$$

Ausklammern:

$$= \frac{\varphi^{n-2} \cdot (1+\varphi) - \psi^{n-2} \cdot (1+\psi)}{\varphi - \psi}$$

Wir nutzen, dass $\varphi^2 = \varphi + 1$ und $\psi^2 = \psi + 1$.

$$= \frac{\varphi^{n-2} \cdot \varphi^2 - \psi^{n-2} \cdot \psi^2}{\varphi - \psi}$$

$$= \frac{\varphi^n - \psi^n}{\varphi - \psi} \quad \blacksquare$$

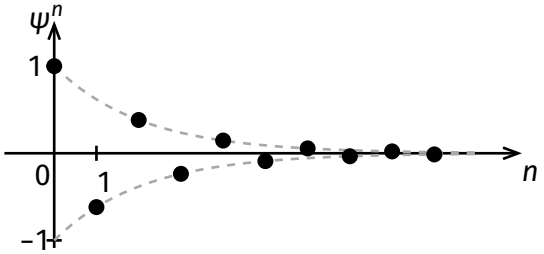
Vereinfachung

Die Formel von Binet besteht aus einem Minuend und einem Subtrahend.

$$\frac{\varphi^n}{\sqrt{5}} - \frac{\psi^n}{\sqrt{5}}$$

(Per Definition: $\varphi - \psi = \sqrt{5}$)

Der Term ψ^n konvergiert gegen null und ist bereits ab $n \geq 0$ im Intervall $[-1, 1]$.



Der Subtrahend verändert das Ergebnis also um weniger als $\frac{1}{2}$.

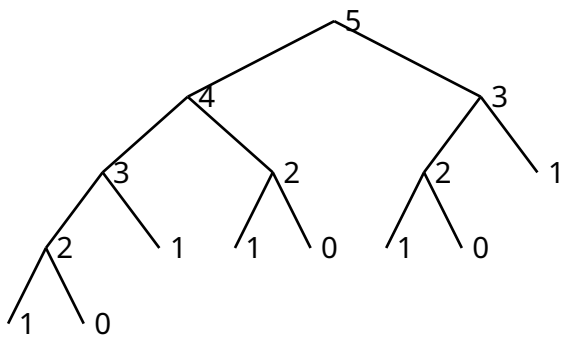
$$\left|\frac{\psi^n}{\sqrt{5}}\right| \leq \frac{1}{\sqrt{5}} < \frac{1}{2}$$

Dieser Fehler wird durch Rundung korregiert, weil die Formel von Binet eine ganze Zahl liefert. Es gilt also die vereinfachte Formel von Binet:

$$\frac{\varphi^n - \psi^n}{\varphi - \psi} = \left\lfloor \frac{\varphi^n}{\varphi - \psi} \right\rceil \quad \forall n \in \mathbb{N}_0$$

Laufzeit

Die Laufzeit der rekursiven Berechnug einer Fibonacci-Zahl f_n ist exponentiell, weil jeder Funktionsaufruf zwei weitere Aufrufe tätigt (f_{n-1} und f_{n-2}), bis der base case erreicht ist.



Sie ist aber schneller als $\Theta(2^n)$, weil der rechte Teilbaum eines Knotens jeweils das Problem um 2 reduziert, statt nur um 1.

Wir wissen, dass die Laufzeit von $T(n)$ für alle $n > 1$ die Summe der beiden Laufzeiten $T(n-1)$ und $T(n-2)$ ist.

$$\begin{aligned}
& T(n) = T(n-1) + T(n-2) \\
\Leftrightarrow \quad & x^n = x^{n-1} + x^{n-2} \\
\Leftrightarrow \quad & \frac{x^n}{x^{n-2}} = \frac{x^{n-1}}{x^{n-2}} + \frac{x^{n-2}}{x^{n-2}} \\
\Leftrightarrow \quad & x^2 = x + 1 \\
\Leftrightarrow \quad & 0 = x^2 - x - 1 \\
\Leftrightarrow \quad & x = \frac{1 \pm \sqrt{5}}{2}
\end{aligned}$$

Das Ergebnis kann nur positiv sein, somit ist

$$T(n) = \left(\frac{1+\sqrt{5}}{2}\right)^n = \varphi^n \quad \blacksquare$$

2.4 O-Notation

Aussage 1

$$\sum_{k=0}^{\infty} \frac{k^2}{2^k} = O(1)$$

Beweis

Die gegebene Reihe ist konvergent. Beweis mittels Wurzelkriterium:

$$\lim_{k\to\infty} \sqrt[k]{\frac{k^2}{2^k}} = \frac{1}{2} \lim_{k\to\infty} \sqrt[k]{k} \cdot \sqrt[k]{k} = \frac{1}{2} \quad \blacksquare$$

Aussage 2

$\forall m \in \mathbb{N}, \alpha > 1 : n^m = O(\alpha^n)$

Beweis

Eine Funktion, die eine andere asymptotisch dominiert, beschränkt sie auch nach oben.

$$\lim_{n\to\infty} \frac{g(n)}{f(n)} \leq c \Rightarrow g(n) = O(f(n))$$

Wir wenden die Regel von de L'Hospital m-mal an, um den Grenzwert zu berechnen.

$$\begin{aligned}
\lim_{n\to\infty} \frac{n^m}{\alpha^n} &\overset{f^{(1)}}{=} \lim_{n\to\infty} \frac{m \cdot n^{m-1}}{\alpha^n \cdot \ln(\alpha)} \\
&\overset{f^{(2)}}{=} \lim_{n\to\infty} \frac{m \cdot (m-1) \cdot n^{m-2}}{\alpha^n \cdot (\ln(\alpha))^2} \\
&\overset{f^{(3)}}{=} \lim_{n\to\infty} \frac{m \cdot (m-1) \cdot (m-2) \cdot n^{m-3}}{\alpha^n \cdot (\ln(\alpha))^3} \\
&\overset{f^{(m)}}{=} \lim_{n\to\infty} \frac{m! \cdot n^{m-m}}{\alpha^n \cdot (\ln(\alpha))^m} \\
&= \frac{m!}{(\ln(\alpha))^m} \cdot \lim_{n\to\infty} \frac{1}{\alpha^n} \\
&= 0 \quad \blacksquare
\end{aligned}$$

Alternativ verwenden wir die e-Funktion. Eine Exponentialfunktion α^n kann als unendliche Reihe dargestellt werden:

$$\begin{aligned}
\alpha^n &= e^{\ln(\alpha^n)} \\
&= e^{n \ln \alpha}
\end{aligned}$$

$$= \sum_{k=0}^{\infty} \frac{(n \ln \alpha)^k}{k!}$$

Weil das eine unendliche Summe ist, gibt es einen Summanden mit $k = m$:

$$\sum_{k=0}^{\infty} \frac{(n \ln \alpha)^k}{k!} = \ldots + \frac{(n \ln \alpha)^m}{m!} + \ldots$$

Ein Summand einer positiven Reihe ist kleiner oder gleich der gesamten Reihe:

$$\frac{(n \ln \alpha)^m}{m!} \leq \underbrace{\sum_{k=0}^{\infty} \frac{(n \ln \alpha)^k}{k!}}_{=\alpha^n}$$

Umformen ergibt:

$$\underbrace{\frac{(\ln \alpha)^m}{m!}}_{\text{Konstant}} \cdot n^m \leq \alpha^n \quad \blacksquare$$

Aussage 3

$$n \ln n = O\left(n^{\frac{3}{2}}\right)$$

Beweis

$$\begin{aligned}
\lim_{n\to\infty} \frac{n \ln n}{n^{\frac{3}{2}}} &= \lim_{n\to\infty} \frac{n}{n} \cdot \frac{\ln(n)}{\sqrt{n}} \\
&= \lim_{n\to\infty} \frac{\frac{1}{n}}{\frac{1}{2\sqrt{n}}} \quad \text{de L'Hospital} \\
&= \lim_{n\to\infty} \frac{2\sqrt{n}}{n} \\
&= \lim_{n\to\infty} \frac{2}{\sqrt{n}} \\
&= 0 \quad \blacksquare
\end{aligned}$$

Aussage 4

$$5^{\log_3 n} = O(n^2)$$

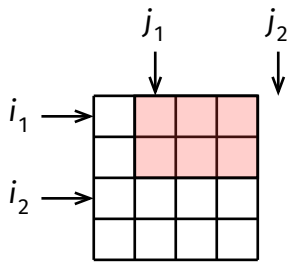
Beweis

$$\begin{aligned}
5^{\log_3 n} &< 9^{\log_3 n} \\
&= 3^{\log_3 n} \cdot 3^{\log_3 n} \\
&= n^2 \quad \blacksquare
\end{aligned}$$

3 Rekursion

3.1 MaxTeilSum-2d

Gesucht ist die Teilmatrix, dessen Komponenten aufsummiert maximal sind.



Naiver Algorithmus

Man könnte durch alle Teilmatrizen iterieren und die Summen vergleichen. Die Laufzeit ist $\Theta(n^6)$, weil es n^4 Kombinationen gibt, welche in n^2 Zeit summiert werden

```
for i2 in range(n+1):
  for i1 in range(i2):
    for j2 in range(n+1):
      for j1 in range(j2):
        matrix[i1:i2, j1:j2].sum()
```

Mit Kadane

Der Algorithmus von Kanade findet die maximale Teilsumme eines Arrays in linearer Laufzeit. Die For-Loops mit Zähler i_1 und i_2 iterieren durch alle möglichen Zeilenkombinationen. Summieren wir die Zeilen von i_1 bis i_2 spaltenweise auf, erhalten wir ein Array, das der Algorithmus von Kanade verarbeiten kann.

```
for i2 in range(n+1):
  for i1 in range(i1):
    yield np.sum(mtx[i1:i2], axis=0)
```

Die Laufzeit ist nur noch $\Theta(n^3)$, weil es n^2 Zeilenkombinationen gibt, welche jeweils mit dem linearen Algorithmus von Kadane verarbeitet werden. Gesamtes Programm: Abschnitt 3.1

3.2 Iteration und Substitution

Gleichung 1

$$T(n) = \begin{cases} 1 \text{ falls } n = 1 \\ 4T\left(\frac{n}{2}\right) + n \text{ sonst} \end{cases}$$

Iteration

$$\begin{aligned}
T(n) &= 4T\left(\frac{n}{2^1}\right) + \frac{n}{2^0} \\
&= 4\left(4T\left(\frac{n}{2^2}\right) + \frac{n}{2^1}\right) + \frac{n}{2^0} \\
&= 4\left(4\left(4T\left(\frac{n}{2^3}\right) + \frac{n}{2^2}\right) + \frac{n}{2^1}\right) + \frac{n}{2^0} \\
&= 4\left(4\left(4\left(4T\left(\frac{n}{2^4}\right) + \frac{n}{2^3}\right) + \frac{n}{2^2}\right) + \frac{n}{2^1}\right) + \frac{n}{2^0} \\
&= \dots \\
&= 4^i T\left(\frac{n}{2^i}\right) + \sum_{k=0}^{i} \frac{n}{2^k}
\end{aligned}$$

Wobei i die Rekursionstiefe ist. Der Base-Case ist bei einer Eingabelänge von 1 erreicht.

$$\begin{aligned}
\frac{n}{2^i} &= 1 \\
n &= 2^i \\
\log_2 n &= i
\end{aligned}$$

Der Base-Case ist also nach $\log_2 n$ Rekursionen erreicht.

$$\begin{aligned}
T(n) &= 4^{\log_2 n} T\left(\frac{n}{2^{\log_2 n}}\right) + \sum_{k=0}^{\log_2 n} \frac{n}{2^k} \\
&= \left(2^{\log_2 n}\right)^2 T(1) + n \sum_{k=0}^{\log_2 n} \left(\frac{1}{2}\right)^k \\
&= n^2 + n \cdot \frac{1 - \left(\frac{1}{2}\right)^{\log_2 n + 1}}{1 - \frac{1}{2}} \\
&= n^2 + n \cdot 2 \cdot \left(1 - \frac{1}{2 \cdot 2^{\log_2 n}}\right) \\
&= n^2 + n\left(2 - \frac{1}{n}\right) \\
&= n^2 + 2n - 1 \\
&= \Theta\left(n^2\right)
\end{aligned}$$

Beweis

Zu zeigen:

$T(n) = \Theta\left(n^2\right)$

Induktionsanfang:

$T(1) = 1 = 1^2$ ✓

Induktionsschritt:

$$\begin{aligned}
T(n) &= \underbrace{4\,T\left(\frac{n}{2}\right)}_{=\Theta(n^2)} + n \\
&= 4n^2 + n \\
&= \Theta\left(n^2\right) \checkmark
\end{aligned}$$

Gleichung 2

$$T(n) = \begin{cases} 1 \text{ falls } n = 1 \\ 2T\left(\frac{n}{4}\right) + \sqrt{n} \text{ sonst} \end{cases}$$

Iteration

$$\begin{aligned}
T(n) &= 2T\left(\frac{n}{4^1}\right) + \sqrt{\frac{n}{4^0}} \\
&= 2\left(2T\left(\frac{n}{4^2}\right) + \sqrt{\frac{n}{4^1}}\right) + \sqrt{\frac{n}{4^0}} \\
&= 2\left(2\left(2T\left(\frac{n}{4^3}\right) + \sqrt{\frac{n}{4^2}}\right) + \sqrt{\frac{n}{4^1}}\right) + \sqrt{\frac{n}{4^0}} \\
&= \dots \\
&= 2^i T\left(\frac{n}{4^i}\right) + \sum_{k=0}^{i-1} 2^k \sqrt{\frac{n}{4^k}} \\
&= 2^{\log_4 n} T(1) + \sum_{k=1}^{\log_4 n} 2^k \sqrt{\frac{n}{4^k}} \\
&= \sqrt{n} + \sum_{k=1}^{\log_4 n} 2^k \cdot \left(\frac{1}{2}\right)^k \cdot \sqrt{n} \\
&= \sqrt{n} + \sum_{k=1}^{\log_4 n} 1^k \cdot \sqrt{n} \\
&= \sqrt{n} + \log_4 n \cdot \sqrt{n} \\
&= \Theta\left(\sqrt{n} \log n\right)
\end{aligned}$$

Gleichung 3

$$T(n) = \begin{cases} 1 \text{ falls } n \in \{1,2,3\} \\ 2T(n-1) + n^2 \text{ sonst} \end{cases}$$

Iteration

$$\begin{aligned} T(n) &= 2T(n-1) + n^2 \\ &= 2\left(2T(n-1) + n^2\right) + n^2 \\ &= 2\left(2\left(2T(n-1) + n^2\right) + n^2\right) + n^2 \\ &= \dots \\ &= 2^i T(n-i) + 2^{i-1} n^2 \\ &= 2^{n-1} T(1) + 2^{n-1} n^2 \\ &= 2^{n-1} \cdot \left(1 + n^2\right) \\ &= \Theta(2^n) \end{aligned}$$

Beweis

Induktionsanfang:

$$T(1) = 1 = \frac{1}{2} \cdot 2^1 \checkmark$$

Induktionsschritt:

$$\begin{aligned} T(n) &= 2\underbrace{T(n-1)}_{=\Theta(2^n)} + n^2 \\ &= 2 \cdot 2^{n-1} + n^2 \\ &= \Theta(2^n) \checkmark \end{aligned}$$

3.3 Master-Methode

Gleichung 1

$$T(n) = \begin{cases} 1 \text{ falls } n = 1 \\ T\left(\frac{n}{2}\right) + 1 \text{ sonst} \end{cases}$$

Zu zeigen

$$T(n) = \Theta(\log n)$$

Beweis

- $a = 1$
- $b = 2$
- $f(n) = 1$

$$\begin{aligned} &\Rightarrow n^{\log_b a} = n^0 = 1 \\ &\Rightarrow \quad f(n) = \Theta\left(n^{\log_b a}\right) \\ &\Rightarrow \quad T(n) = \Theta\left(n^{\log_b a} \log n\right) \\ &\qquad\qquad = \Theta(\log n) \blacksquare \end{aligned}$$

Gleichung 2

$$T(n) = \begin{cases} 1 \text{ falls } n = 1 \\ 3T\left(\frac{n}{4}\right) + n \log n \text{ sonst} \end{cases}$$

Zu zeigen

$$T(n) = \Theta(n \log n)$$

Beweis

- $a = 3$
- $b = 4$
- $f(n) = n \log n$
- $\varepsilon := 1 - \log_4 3$

$$\begin{aligned} &\Rightarrow \quad n^{\log_b a + \varepsilon} = n \\ &\Rightarrow \lim_{n \to \infty} \frac{f(n)}{n^{\log_b a + \varepsilon}} = \infty \\ &\Rightarrow \quad f(n) = \Omega\left(n^{\log_b a + \varepsilon}\right) \end{aligned}$$

Zusätzlich muss gelten:

$$\begin{aligned} & a \cdot f\left(\frac{n}{b}\right) \le c \cdot f(n) \\ \Leftrightarrow \quad & 3 \cdot \frac{n}{4} \log \frac{n}{4} \le c \cdot n \log n \\ \Leftrightarrow \quad & 3 \cdot \frac{\frac{n}{4} \log \frac{n}{4}}{n \log n} \le c \\ \Leftrightarrow \quad & \frac{3}{4} \log_n \frac{n}{4} \le c \\ \Leftrightarrow \quad & \frac{3}{4} \underbrace{\left(1 - \log_n 4\right)}_{\le 1} \le c \checkmark \end{aligned}$$

$$\begin{aligned} \Rightarrow T(n) &= \Theta(f(n)) \\ &= \Theta(n \log n) \blacksquare \end{aligned}$$

Gleichung 3

$$T(n) = \begin{cases} 1 \text{ falls } n = 1 \\ 7T\left(\frac{n}{2}\right) + n^2 \text{ sonst} \end{cases}$$

Zu zeigen

$$T(n) = \Theta\left(n^{2.81}\right)$$

Beweis

- $a = 7$
- $b = 2$
- $f(n) = n^2$
- $\varepsilon := \log_2(7) - 2$

$$\Rightarrow n^{\log_b a-\varepsilon} = n^2$$
$$\Rightarrow \quad f(n) = \Theta\left(n^{\log_b a-\varepsilon}\right)$$
$$\Rightarrow \quad T(n) = \Theta\left(n^{\log_b a}\right)$$
$$\approx \Theta\left(n^{2.81}\right) \blacksquare$$

3.4 Ackermannfunktion

$$f(n,m) = \begin{cases} m+1 \text{ falls } n = 0 \\ f(n-1,1) \text{ falls } m = 0 \text{ und } n \geq 1 \\ f(n-1, f(n, m-1)) \text{ sonst} \end{cases}$$

Rekursiver Algorithmus

```
def ack(n, m):
    if n == 0:
        return m+1

    if m == 0 and n ≥ 1:
        return ack(n-1, 1)

    return ack(n-1, ack(n, m-1))
```

Iterativer Algorithmus

```
stack = deque(((n, m),))
result=0

while stack:
  n, m = stack.pop()
  if m is None:
    m = result

  if n == 0:
    result = m+1
    continue

  if m == 0:
    stack.append((n-1, 1))
    continue

  stack.append((n-1, None))
  stack.append((n, m-1))

return result
```

Korrektheit

Um zu zeigen, dass $f(n,m)$ für alle Eingaben $(n,m) \in \mathbb{N}_0^2$ ein Ergebnis liefert, verwenden wir eine Induktion über zwei Variablen [15].

Zu zeigen

$\forall n, m \in \mathbb{N}_0 : f(n,m)$ ist definiert

Beweis

Wir zeigen, dass f für alle n definiert ist, indem wir zeigen, dass

$f(n-1,m)$ definiert $\Rightarrow f(n,m)$ definiert

Für jedes n zeigen wir mittels verschachtelter Induktion, dass f für dieses n und für jedes $m \in \mathbb{N}_0$ definiert ist.

$f(n,m-1)$ definiert $\Rightarrow f(n,m)$ definiert

Induktionsanfang (n = 0)

Es gilt per Definition für $n = 0$ und alle $m \in \mathbb{N}_0$:

$f(0,m) = m + 1$ ✓

Induktionsschritt (n - 1 ⟶ n)

Wir nehmen an, dass $f(n-1,m)$ für ein $n-1$ und alle $m \in \mathbb{N}_0$ definiert ist. Das dürfen wir, weil wir im Basisfall gezeigt haben, dass $f(0,m)$ für alle $m \in \mathbb{N}_0$ definiert ist.

Anhand dieser Annahme möchten wir zeigen, dass $f(n,m)$ für n und alle $m \in \mathbb{N}_0$ definiert ist.

Verschachtelte Induktion

Induktionsanfang (m = 0)

$f(n,0) = f(n-1,1)$ [Def.]

Nach Induktionsannahme ist f für ein $n-1$ und alle $m \in \mathbb{N}_0$, insbesondere $m = 1$ definiert. ✓

Induktionsschritt (m - 1 ⟶ m)

Wir nehmen an, dass $f(n, m-1)$ definiert ist, und folgern daraus, dass $f(n, m)$ definiert ist.

$$f(n,m) = f(n-1, \underbrace{f(n, m-1)}_{=:x \in \mathbb{N}_0 \text{ (Nach IV)}}) \text{ [Def.]}$$
$$= f(n-1, x)$$

Nach innerer Induktionsannahme ist $f(n, m-1) =: x$ definiert. Nach äußerer Induktionsannahme ist $f(n-1, x)$ für alle $x \in \mathbb{N}_0$ definiert. ✓

Induktionsschluss

Wir haben gezeigt, dass $f(n, m)$ für alle m definiert ist, sofern $f(n-1, m)$ für alle m definiert ist. □

Induktionsschluss

Wir haben gezeigt, dass $f(n, m)$ für alle n definiert ist, sofern $f(n-1, m)$ definiert ist. In der verschachtelten Induktion haben wir für alle n gezeigt, dass $f(n, m)$ auch für alle m definiert ist. ■

4 Sortieralgorithmen

4.1 Sortieralgorithmen modifizieren

Insertion Sort

Statt dass in jedem Schleifendurchlauf die Elemente links des Schlüssels sortiert sind, sollten jetzt jene rechts des Schlüssels sortiert sein. Um das zu bewerkstelligen, drehen wir die äußere und innere Iteration um. Die äußere verläuft jetzt nach links und die innere nach rechts.

```
for j in range(n):
  key = a[j]
  i = j-1
  while i ⩾ 0 and a[i] > key:
    a[i+1] = a[i]
    i -= 1
  a[i+1] = key
```

↓

```
for j in reversed(range(n)):
  key = a[j]
  i = j+1
  while i < n and a[i] < key:
    a[i-1] = a[i]
    i += 1
  a[i-1] = key
```

Laufzeit & Korrektheit

Die Laufzeit bleibt gleich. Der Korrektheitsbeweis erfolgt analog zum Beweis des ursprünglichen Algorithmus.

Mehr über Insertion Sort: Abschnitt 4.1

Bubble Sort

Um das Maximum nach hinten wandern zu lassen, drehen wir wieder beide Schleifen um.

```
for i in range(n):
  for j in reversed(range(i, n-1))
   if a[j] > a[j+1]:
      a[j], a[j+1] = a[j+1], a[j]
```

↓

```
for i in reversed(range(n)):
  for j in range(i, n-1):
   if a[j] > a[j+1]:
      a[j], a[j+1] = a[j+1], a[j]
```

Laufzeit & Korrektheit

Entspricht dem ursprünglichen Algorithmus.

Mehr über Bubble Sort: Abschnitt 4.2

Selection Sort

```
for i in range(n):
```

```
  x = a.index(min(a[i:]))
  a[i], a[x] = a[x], a[i]
```

↓

```
for i in reversed(range(n)):
  x = a.index(max(a[:i+1]))
  a[i], a[x] = a[x], a[i]
```

Laufzeit & Korrektheit

Entspricht dem ursprünglichen Algorithmus.

Mehr über Selection Sort: Abschnitt 4.3

Quicksort

Anstatt einen zufälligen Index zu wählen, können wir einfach `a[f]` mit einer zufälligen Zahl vertauschen.

```
def partition(a, f, l):
  # ...
```

↓

```
def partition(a, f, l):
  i = randint(f, l)
  a[f], a[i] = a[i], a[f]
  # ...
```

Laufzeit & Korrektheit

Es wird mal pro `partition`-Aufruf mit $\Theta(1)$ zufällig vertauscht, damit bleibt die Laufzeit unverändert.

Wenn die nicht-randomisierte Partition korrekt ist, dann ist die Randomisierte auch korrekt, weil die Eingabe `a[f:l+1]` in beliebiger Permutation auftreten darf. Die Reihenfolge der Elemente hat keinen Einfluss auf die Funktionsweise.

4.2 Zeitmessungen

Zufällige Zahlenfolge der Länge n:

```
[randint(-5, 256) for _ in range(n)]
```

Um die maximale Eingabelänge zunächst grob zu ermitteln, prüfen wir jede Potenz von 2 (`max_input.py`).

```
for n in (2**i for i in count()):
  if runtime > 60:
    return n
```

Das Infimum der Eingabelängen muss also in $\text{lo} := \frac{n}{2}$ und $\text{hi} := n$ liegen. Mit der Intervallhalbierungsmethode können wir dieses n zu einer beliebigen Genauigkeit `depth` berechnen (`exact_input.py`).

```
for _ in range(depth):
  m = (hi + lo) // 2
  if runtime > 60:
    hi = m
  else:
    lo = m
return lo
```

Mit einer Tiefe von 20 erhalte ich folgende Eingabelängen n, welche innerhalb von $t < 60s$ berechnet werden können:

	Bubblesort		Quicksort		Mergesort	
	n	t	n	t	n	t
lo	2^{14}	$50s$	2^{17}	$20s$	2^{22}	$57s$
hi	2^{15}	$> 60s$	2^{18}	$0.3s$	2^{23}	$> 60s$
=	$24K$	$46s$	$229K$	$24s$	$8M$	$46s$

Quicksort überschreitet die maximale Rekursionstiefe, bevor die Zeitgrenze von einer Minute erreicht wird.

4.3 Minkowski-Summe

Gefragt ist, ob es ein $s \in \mathbb{Z}$ gibt, welches sich aus der Summe von zwei Elementen $a_i, a_j \in A$ ergibt.

$$s \in \left\{a_i + a_j \,\middle|\, (a_i, a_j) \in A \times A, i \neq j\right\}$$

Naiver Ansatz

Für alle $\left\{a_i + a_j \,\middle|\, (a_i, a_j) \in A \times A, i \neq j\right\}$ prüfen, ob $a_i + a_j = s$.

```
from itertools import combinations
```

```
for ai, aj in combinations(A, 2):
  if ai + aj == s: return True
return False
```

Besserer Ansatz

Wählen wir $a_i = \min(A)$ und $a_j = \max(A)$, dann ist die nächstgrößere Summe

$$\text{sum}_> = \min(A \setminus \{a_i\}) + a_j$$

und die nächstkleinere Summe

$$\text{sum}_< = a_i + \max(A \setminus \{a_j\})$$

Wenn A sortiert ist, dann lässt sich der nächste Wert in $\Theta(1)$ berechnen:

$$\text{sum}_> = a_{i+1} + a_j$$

$$\text{sum}_< = a_i + a_{j-1}$$

Wir starten mit $l = 0$ und $r = n - 1$, und laufen zur Mitte.

```
a.sort()
l = 0
r = len(a) - 1
while l < r:
  sum = a[l] + a[r]
  if sum == s: return True
  if sum < s: l += 1
  else: r -= 1
return False
```

Das sortieren dauert $\Theta(n \log n)$, damit dauert der gesamte Algorithmus $\Theta(n \log n)$.

4.4 Iterativ/Rekursiv

Insertion Sort Rekursiv

In jedem Rekursionsschritt wird die Eingabelänge n um 1 reduziert.

```
def insertion_sort(a: list, n: int):
  if n ≤ 1: return

  insertion_sort(a, n-1)
  for i in range(n):
    if a[n-1] < a[i]:
      a.insert(i, a.pop(n-1))
      return
```

Merge Sort Iterativ

Die Merge-Funktion bleibt unberührt. Für jedes $i \in (2, 4, 8, \ldots, n)$ wird die Eingabe in mehrere i-Elementige Teillisten gespaltet.

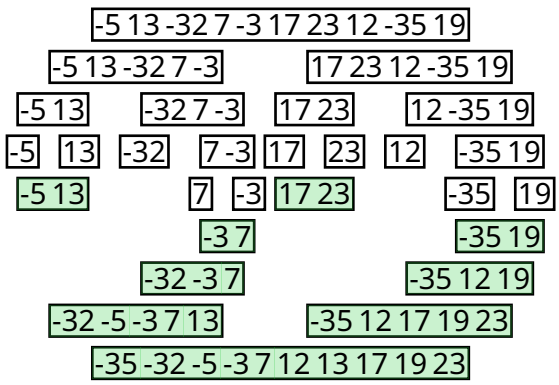
```
for i in (2 ** i for i in count(1)):
  for j in range(0, len(a), i):
    f = j
    l = min(len(a)-1, j+i-1)
    # merge a[f:l+1]
  if i ≥ len(a):
    break
```

5 Heaps

5.1 Mergesort und Heapsort

Mergesort

Rekursionsbaum



In Python

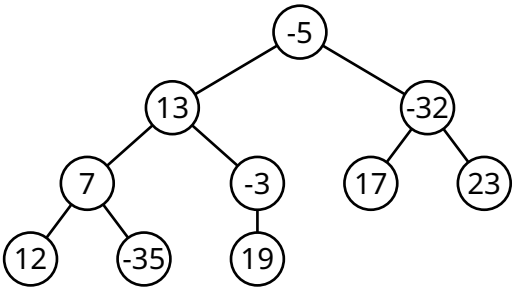
```
def merge(a, i, j, l):
  while i < j and j ≤ l:
    if a[j] < a[i]:
      a.insert(i, a.pop(j))
      j += 1
    i += 1
```

```python
def mergesort(a, f, l):
  if f ≥ l: return
  m=(f+l)//2
  mergesort(a, f, m)
  mergesort(a, m+1, l)
  merge(a, f, m+1, l)

a=[-5, 13, -32, 7, -3, 17, 23, 12, -35, 19]
mergesort(a, 0, len(a)-1)
print(a)
```

Heapsort

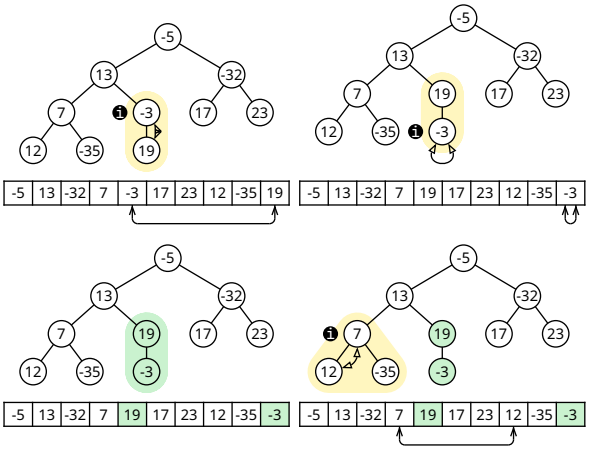
Demonstration

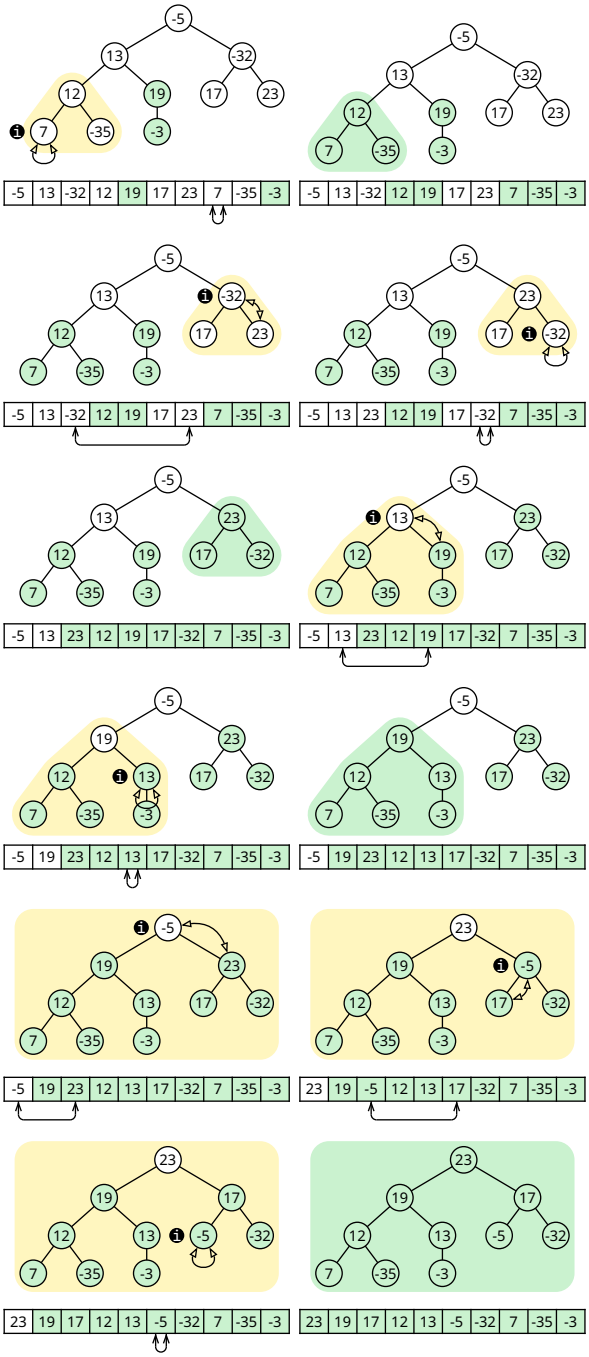
0. Eingabe



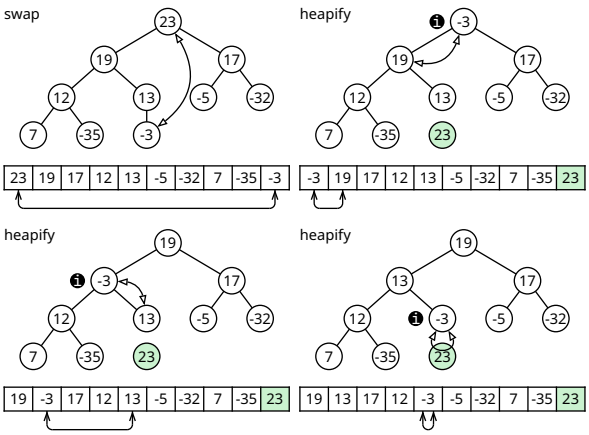
-5	13	-32	7	-3	17	23	12	-35	19

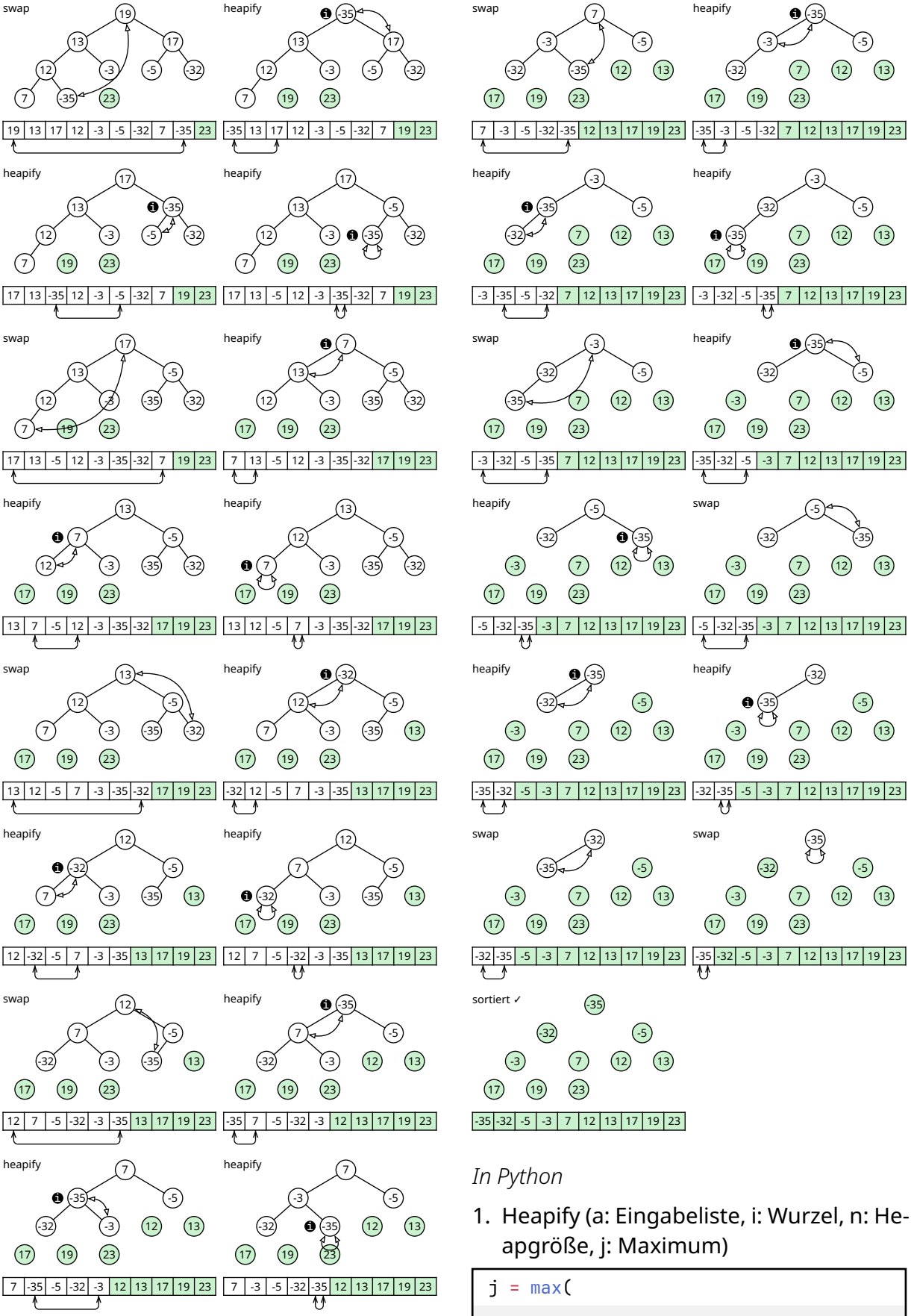
1. BuildHeap





2. Sortieren





In Python

1. Heapify (a: Eingabeliste, i: Wurzel, n: Heapgröße, j: Maximum)

```
j = max(
    i, 2*i+1, 2*i+2,
    key=lambda i:
```

```
    a[i] if i < n else -math.inf
)
if i ≠ j:
  a[i], a[j] = a[j], a[i]
  heapify(a, j, n)
```

2. BuildHeap (a: Eingabeliste)

```
for i in range(len(a)//2-1, -1, -1):
  heapify(a, i, len(a))
```

3. HeapSort (a: Eingabeliste)

```
build_heap(a)
for i in range(len(a)-1, 0, -1):
  a[0], a[i] = a[i], a[0]
  heapify(a, 0, i)
```

5.2 BuildHeap Laufzeit Aussagen

Aussage 1

Ein Heap mit n Elementen hat die Höhe $\lfloor \log_2(n) \rfloor$.

Beweis

Weil jeder Knoten zwei Nachfolger haben darf, hat jede Schicht doppelt so viele Knoten wie die vorherige. Ein voll besetzter Heap mit Höhe h hat so viele Elemente:

$$n = 1 + 2 + 4 + 8 + \ldots + 2^h$$

Diese geometrische Reihe lässt sich umschreiben:

$$n = \sum_{i=0}^{h} 2^i = 2^{h+1} - 1$$

Aber nicht jeder Heap ist voll besetzt, weil die tiefste Schicht nicht voll besetzt sein muss. Die genaue Anzahl an Elementen liegt also zwischen 2^h und $2^{h+1} - 1$.

Daraus folgt, dass umgekehrt die Höhe mindestens $\lfloor \log n \rfloor$ ist.

$$\begin{aligned} & n \geq 2^h \\ \Leftrightarrow\ & \log n \geq h \\ \Rightarrow\ & \lfloor \log n \rfloor \geq h \end{aligned}$$

Sie ist auch höchstens $\lfloor \log n \rfloor$.

$$\begin{aligned} & n \leq 2^{h+1} - 1 \\ \Leftrightarrow\ & n < 2^{h+1} \\ \Leftrightarrow\ & \log n < h + 1 \\ \Rightarrow\ & \lfloor \log n \rfloor < h + 1 \\ \Leftrightarrow\ & \lfloor \log n \rfloor \leq h \end{aligned}$$

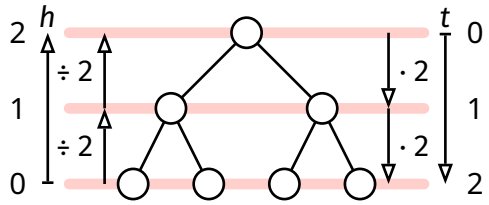
Damit gilt $\lfloor \log n \rfloor = h$ ■

Aussage 2

Ein Heap mit n Elementen hat höchstens $\left\lceil \frac{n}{2^{h+1}} \right\rceil$ Knoten auf Höhe h.

Idee für vollständige Heaps

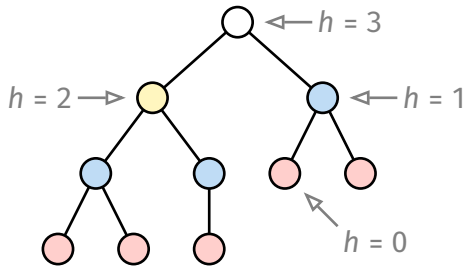
Ein vollständiger Heap hat in jeder Schicht doppelt so viele Knoten wie in der vorherigen, weil jeder Knoten bis zu 2 Nachfolger hat. Umgekehrt hat jede Schicht halb so viele Knoten wie die nächste Schicht.



Es gibt auf einer Schicht mit Tiefe t 2^t Knoten. Umgekehrt gibt es auf einer Schicht mit Höhe h 0.5^h mal so viele Knoten wie auf der untersten Schicht.

Idee für beliebige Heaps

Die Höhe eines Knotens entspricht nicht der Schicht, sondern dem längsten Abstand zu einem Blatt.



Um die obige Formel dennoch zu zeigen, gehen wir induktiv vor: Im Induktionsanfang zeigen wir, dass es immer höchstens $\lceil n/2 \rceil$ Blätter in einem Heap gibt. Im Induk-

tionsschritt nutzen wir, dass sich die Höhe aller Knoten in einem Baum um 1 reduziert, wenn wir seine Blätter entfernen.

Induktionsanfang (h = 1)

Der Vorgänger des letzten Blattes $n - 1$ ist

$$\left\lfloor \frac{n-1-1}{2} \right\rfloor = \left\lfloor \frac{n}{2} \right\rfloor - 1$$

Die Anzahl an nicht-Blättern ist der Index plus 1.

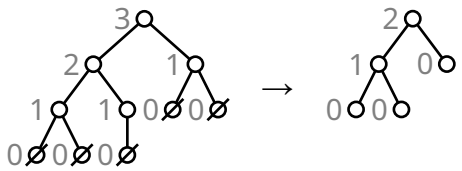
$$\left\lfloor \frac{n}{2} \right\rfloor - 1 + 1 = \left\lfloor \frac{n}{2} \right\rfloor$$

Die Anzahl an Blättern ist die Gesamtknotenzahl minus die nicht-Blätter.

$$n - \left\lfloor \frac{n}{2} \right\rfloor = \left\lceil \frac{n}{2} \right\rceil \checkmark$$

Induktionsschritt (h - 1 ⟶ h)

Wir haben gezeigt, dass jeder Heap H $\lceil n/2 \rceil$ Blätter hat. Wenn wir diese entfernen, bleibt ein Heap H' mit $\lfloor n/2 \rfloor$ Einträgen übrig. Die Höhe der Knoten in H' ist um 1 geringer als die in H.



Nach Induktionsannahme gilt: Ein Heap hat

$$\left\lceil \frac{n}{2^{(h-1)+h}} \right\rceil = \left\lceil \frac{n}{2^h} \right\rceil$$

Knoten der Höhe $h - 1$. Wir setzen die Anzahl der Knoten aus H' ein, um die Knoten auf Höhe $h - 1$ von H' zu erhalten:

$$\left\lceil \frac{\lfloor n/2 \rfloor}{2^h} \right\rceil$$

Die Knoten auf Höhe $h - 1$ von H' sind die Knoten auf Höhe h von H. Wir führen die Formel zurück auf die zu zeigende:

$$\left\lceil \frac{\lfloor n/2 \rfloor}{2^h} \right\rceil \leq \left\lceil \frac{n/2}{2^h} \right\rceil = \left\lceil \frac{n}{2^{h+1}} \right\rceil \blacksquare$$

Aussage 3

Für alle $-1 < x < 1$ gilt

$$\sum_{k=0}^{\infty} kx^k = \frac{x}{(1-x)^2}$$

Beweis

Die unendliche geometrische Reihe mit Basis $|q| < 1$ ist

$$\sum_{k=0}^{\infty} q^k = \frac{1}{1-q}$$

Beidseitiges differenzieren liefert

$$\sum_{k=0}^{\infty} kq^{k-1} = \frac{1}{(1-q)^2}$$

Wir multiplizieren q hinzu

$$\sum_{k=0}^{\infty} kq^{k-1} \cdot q = \frac{1}{(1-q)^2} \cdot q$$

$$\Leftrightarrow \sum_{k=0}^{\infty} kq^k = \frac{q}{(1-q)^2} \blacksquare$$

5.3 Strassen

Um die Matrixmultiplikation rekursiv zu berechnen, vierteln wir die Problemgröße. Wir berechnen jeden Quadranten der Ausgabe seperat als $n/2 \times n/2$ Matrix.

$$\begin{bmatrix} M_{11} & M_{12} \\ M_{21} & M_{22} \end{bmatrix} \cdot \begin{bmatrix} N_{11} & N_{12} \\ N_{21} & N_{22} \end{bmatrix} = \begin{bmatrix} O_{11} & O_{12} \\ O_{21} & O_{22} \end{bmatrix}$$

Die vier Ausgabematrizen werden durch die Eingabematrizen folgenderweise berechnet

$$O_{ij} = \sum_{k=1}^{2} M_{ik} \cdot N_{kj}$$

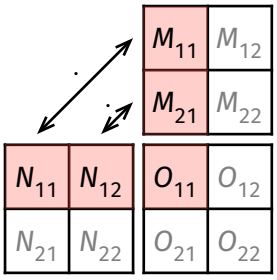
Die Berechnung eines Quadranten teilt das Problem wiederum in vier, bis der Base Case erreicht ist. Das ist, wenn die Eingabematrizen 1×1 Matrizen, also Skalare sind. Dann werden lediglich die Zahlen miteinander multipliziert.

Korrektheitsbeweis Rekursion

Die Multiplikation zweier $n \times n$ Matrizen $M = \left(m_{ij}\right)$ und $N = \left(n_{ij}\right)$ ist per Definition

$$o_{ij} = \sum_{k=1}^{n} m_{ik} n_{kj}$$

Betrachten wir zunächst die Berechnung von O_{11}.



Wir umschreiben die Berechnung so, dass wir uns auf die ganzen Matrizen M und N beziehen. (m_{ij} und n_{ij} sind Einträge aus M und N. o_{ij} sind Einträge aus O_{11}.)

$$O_{11} = M_{11} \cdot N_{11} + M_{12} \cdot N_{21}$$

$$\begin{aligned} o_{ij} &= \sum_{k=1}^{n/2} m_{ik} n_{kj} + \sum_{k=n/2+1}^{n} m_{ik} n_{kj} \\ &= \sum_{k=1}^{n} m_{ik} n_{kj} \ \checkmark \end{aligned}$$

Um O_{22} in Bezug auf M und N umzuschreiben, verschieben wir die Indizes, indem wir definieren $p \coloneqq i - n/2$ und $q \coloneqq j - n/2$.

$$O_{22} = M_{21} \cdot N_{12} + M_{22} \cdot N_{22}$$

$$\begin{aligned} o_{pq} &= \sum_{k=1}^{n/2} m_{ik} n_{kj} + \sum_{k=n/2+1}^{n} m_{ik} n_{kj} \\ &= \sum_{k=1}^{n} m_{ik} n_{kj} \ \checkmark \end{aligned}$$

Das gleiche könnten wir für O_{12} und O_{21} machen, indem wir als Ausgabe o_{pj} und o_{iq} wählen. ■

Korrektheitsbeweis Strassen

Wir zeigen für jeden Quadranten O_{11}, O_{12}, O_{21} und O_{22}, dass er korrekt ist.

Erster Quadrant (11)

Definition abschreiben:

$$\begin{aligned} O_{11} &= H_1 + H_4 - H_5 + H_7 \\ &= \left(M_{11} + M_{22}\right) \cdot \left(N_{11} + N_{22}\right) + \\ &\quad M_{22} \cdot \left(N_{21} - N_{11}\right) - \\ &\quad \left(M_{11} + M_{12}\right) \cdot N_{22} + \\ &\quad \left(M_{12} - M_{22}\right) \cdot \left(N_{21} + N_{22}\right) \end{aligned}$$

Die Matrixmultiplikation ist distributiv; es gilt $A(B + C) = AB + AC$ und $(B + C)D = BD + CD$.

$$\begin{aligned} O_{11} &= M_{11}N_{11} + M_{11}N_{22} + M_{22}N_{11} + M_{22}N_{22} + \\ &\quad M_{22}N_{21} - M_{22}N_{11} - \\ &\quad M_{11}N_{22} - M_{12}N_{22} + \\ &\quad M_{12}N_{21} + M_{12}N_{22} - M_{22}N_{21} - M_{22}N_{22} \end{aligned}$$

Die meisten Terme heben sich gegenseitig auf.

$$O_{11} = M_{11}N_{11} + M_{12}N_{21}$$

Die Korrektheit dieses Termes haben wir oben bewiesen. ✓

Zweiter Quadrant (21)

$$\begin{aligned} O_{21} &= H_3 + H_5 \\ &= M_{11} \cdot \left(N_{12} - N_{22}\right) + \left(M_{11} + M_{12}\right) \cdot N_{22} \\ &= M_{11}N_{12} - M_{11}N_{22} + M_{11}N_{22} + M_{12}N_{22} \\ &= M_{11}N_{12} + M_{12}N_{22} \end{aligned}$$

Ebenfalls oben bewiesen. ✓

Dritter Quadrant (12)

$$\begin{aligned} O_{21} &= H_2 + H_4 \\ &= \left(M_{21} + M_{22}\right) \cdot N_{11} + M_{22} \cdot \left(N_{21} - N_{11}\right) \\ &= M_{21}N_{11} + M_{22}N_{11} + M_{22}N_{21} - M_{22}N_{11} \\ &= M_{21}N_{11} + M_{22}N_{11} \end{aligned}$$

Ebenfalls oben bewiesen. ✓

Vierter Quadrant (22)

$$\begin{aligned} O_{22} &= H_1 - H_2 + H_3 + H_6 \\ &= \left(M_{11} + M_{22}\right) \cdot \left(N_{11} + N_{22}\right) - \end{aligned}$$

$$
\begin{aligned}
&(M_{21} + M_{22}) \cdot N_{11} + \\
&M_{11} \cdot (N_{12} - N_{22}) + \\
&(M_{21} - M_{11}) \cdot (N_{11} + N_{12}) \\
= &M_{11}N_{11} + M_{11}N_{22} + M_{22}N_{11} + M_{22}N_{22} - \\
&M_{21}N_{11} - M_{22}N_{11} + \\
&M_{11}N_{12} - M_{11}N_{22} + \\
&M_{21}N_{11} + M_{21}N_{12} - M_{11}N_{11} - M_{11}N_{12} \\
= &M_{21}N_{12} + M_{22}N_{22}
\end{aligned}
$$

Ebenfalls oben bewiesen. ✓ ■

Laufzeit Rekursion

Der Algorithmus teilt das Problem in acht Probleme mit halber Größe. Das kombinieren der Teilprobleme läuft in quadratischer Zeit, weil komponentenweise Addiert wird. Die Rekursionsgleichung ist also

$$T(n) = 8T\left(\frac{n}{2}\right) + 4\left(\frac{n}{2}\right)^2$$

Im Base-Case werden acht 1×1 Matrizen in konstanter Zeit multipliziert, also ist $T(1) = \Theta(1)$.

Durch den ersten Fall des Master-Theorems ist die Laufzeit $\Theta(n^3)$.

Laufzeit Strassen

Die Strassen-Multiplikation teilt das Problem in nur sieben Teilprobleme mit halber Größe.

$$T(n) = 7T\left(\frac{n}{2}\right) + 18\left(\frac{n}{2}\right)^2$$

Durch den ersten Fall des Master-Theorems ist die Laufzeit

$$T(n) = \Theta\left(n^{\log_2 7}\right) \approx \Theta\left(n^{2.81}\right)$$

Laufzeit Naiver Algorithmus

Der Standardalgorithmus, der aus der Definition hervorgeht, summiert für jede Zelle aus der $n \times n$ Ausgabematrix n Terme auf. Daraus ergibt sich eine Laufzeit von $\Theta(n^3)$.

Der Naive Divide-And-Conquer Algorithmus hat die gleiche Laufzeit, aber der Strassen-Algorithmus ist schneller.

5.4 Strassen Implementierung

Strassen-Algorithmus in Python: (a und b: Eingabematrizen, `quarters`: Viertelt die Eingabe)

```python
# Base Case
if a.shape == (1, 1):
  return a[0, 0] * b[0, 0]


# Divide
a11, a12, a21, a22 = quarters(a)
b11, b12, b21, b22 = quarters(b)


# Conquer
h1 = mult((a11 + a22), (b11 + b22))
h2 = mult((a21 + a22), b11)
h3 = mult(a11, (b12 - b22))
h4 = mult(a22, (b21 - b11))
h5 = mult((a11 + a12), b22)
h6 = mult((a21 - a11), (b11 + b12))
h7 = mult((a12 - a22), (b21 + b22))


# Merge
return np.vstack((
  np.hstack((
    h1 + h4 - h5 + h7,
    h3 + h5
  )),
  np.hstack((
    h2 + h4,
    h1 - h2 + h3 + h6
  ))
))
```

Laufzeit

n	Naiv [s]	Strassen [s]
4	0	0
8	0	0
16	0.1	0
32	0.6	0.2
64	4.5	1.3
128	20.2	9

Die Laufzeitverbesserung des Strassen-Algorithmus ist schon bei minimalen Eingabelängen spürbar.

6 Datenstrukturen

6.1 Sortieren in Linearzeit

Counting Sort

Vergleich Heap-, Count- und Map Sort

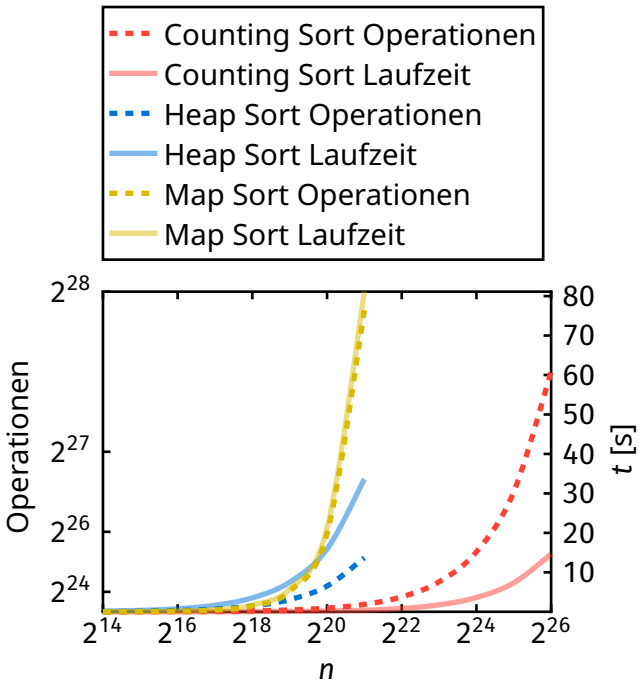
Map Sort

```
if b[i] == -1:
  b[i] = key
  break

if dir == "r" and key < b[i] or \
   dir == "l" and key > b[i]:
  key, b[i] = b[i], key
```

Komplexitätsanalyse

Counting Sort läuft ab $n > 9000$ in Linearzeit, aber benötigt immer mindestens 9.000 Schritte. Heapsort läuft immer in $\Theta(n \log n)$. Map Sort läuft immer in $\Theta(n)$.

Laufzeitmessungen



6.2 Verkettete Liste

Folgende Generator-Funktion traversiert alle Elemente von Head nach Tail. Die Liste darf während der Iteration verändert werden.

```
def nodes(head, tail):
  current = head
  while current:
    h = current.next
    yield current
    if current == tail:
      break
    current = h
```

In der Partition-Funktion werden Elemente kleiner als der Pivot direkt vorgezogen. Dadurch könnten Head und Tail verändert werden, also geben wir sie mit Pivot Zurück.

```
def partition(head, tail):
  pivot = head
  for node in nodes(head.next, tail):
    if node.value > pivot.value:
      continue
```

```
    if node == tail:
      tail = tail.prev
    node.detach()
    head.prepend(node)
    head = node

  return (head, pivot, tail)
```

Nach dem gleichen Prinzip werden Head und Tail in Quicksort stets angepasst.

```
def quicksort(head, tail):
  if head == tail:
    return (head, tail)
  h, p, t = partition(head, tail)

  if p != h:
    h, _ = quicksort(h, p.prev)
  if p != t:
    _, t = quicksort(p.next, t)

  return (h, t)
```

Weil in jeder Rekursion Variablen angelegt werden, ist der Platzverbrauch $\Theta(\log n)$. Die Laufzeit entspricht der des normalen Quicksort-Algorithmus, also $\Theta(n \log n)$.

6.3 Ringliste

Datenstruktur Verkettete Liste:

```
class Node:
  def __init__(self, value: int):
    self.value = value
    self.next = None

  def step(self, n: int):
    c = self
    for _ in range(n):
      c = c.next
    return c

  def pop(self):
    v = self.next.value
    self.next = self.next.next
    return v
```

Initialisierung der Lotto-Liste:

```
head = Node(1)
tail = head
while True:
  if tail.value == 49:
    tail.next = head
    break
  new = Node(tail.value+1)
  tail.next = new
  tail = new
return head
```

Ziehen von sechs Zufallszahlen:

```
l = lotto_list()
for i in range(6):
  steps = random.randint(0, 48-i)
  draw = l.step(steps)
  value = draw.pop()
  yield value
```

6.4 Rucksackproblem

Stetiges Rucksackproblem

Um das anteilige Rucksackproblem zu lösen, sortieren wir zunächst alle Stücke nach Wert pro Kilo:

```
def ratio(item):
    return item.value / item.weight

a=sorted(items, key=ratio,
         reverse=True):
```

Dann packen wir die höherwertigen Stücke nacheinander ein, bis eines ggf. nicht ganz hineinpasst. Dieses schneiden wir dann zurecht, indem wir es einen Faktor kleiner als 1 zuweisen.

```
sack = []
for item in a:
  if item.weight > max_weight:
    sack.append([
      item,
      max_weight / item.weight
    ])
    break
  sack.append([item, 1])
max_weight -= item.weight
return sack
```

Der Algorithmus läuft in $\Theta(n \log n)$, weil das sortieren am längsten dauert.

Diskretes Rucksackproblem

7 BST und AVL-Bäume

7.1 BST

Rekursives Löschen

```
if i < self.root.value: # i is left
  left = BST(self.root.l)
  left.delete(i)
  self.root.l = left.root
elif i > self.root.value: # i is right
  right = BST(self.root.r)
  right.delete(i)
  self.root.r = right.root
else: # i is root
  if not self.root.l:
      self.root = self.root.r
  elif not self.root.r:
      self.root = self.root.l
  else: # root has two children
      right = BST(self.root.r)
      suc = right.min()
      self.root.value = suc.value
      right.delete(suc.value)
      self.root.r = right.root
```

Laufzeit: $O(h)$, um den Knoten zu finden, und $O(h)$, um ihn zu löschen (balancierter Baum).

Iteratives Löschen

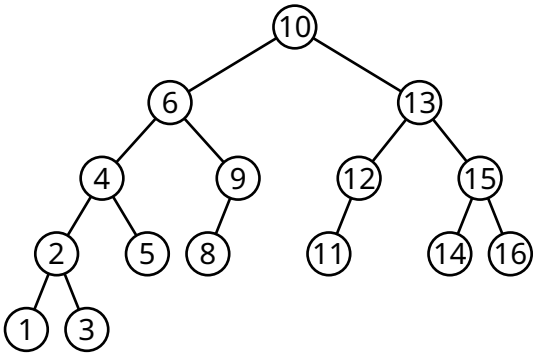
```
dp, d = self.searchp(i)

if not d.r:
  if dp:
    if d.value < dp.value:
      dp.l = d.l
    else:
      dp.r = d.l
  else:
    self.root = d.l
  return

if not d.l:
  if dp:
    if d.value < dp.value:
      dp.l = d.r
    else:
      dp.r = d.r
  else:
    self.root = d.r
  return

sp, s = self.sucp(d)

if s == d.r:
  s.l = d.l
else:
  sp.l = s.r
  s.l = d.l
  s.r = d.r

if dp:
  if d.value < dp.value:
    dp.l = s
```

```
  else:
    dp.r = s
else:
  self.root = s
```

7.2 BST Eindeutigkeit

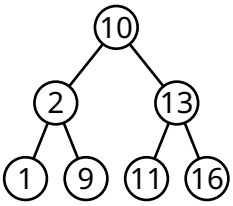
7.3 AVL Einfügen und Löschen

Einfügen




```
if self.root == NullNode():
  self.root = node
elif node.value < root.value:
  if root.l == NullNode():
    root.l = node
  else:
    self.insert(node, root.l)
else:
  if root.r == NullNode():
    root.r = node
  else:
    self.insert(node, root.r)

root.refresh_height()
root.rebalance()
```

Löschen




```
if value < root.value:
  self.delete(value, root.l, root)
elif value > root.value:
  self.delete(value, root.r, root)
else:
  if root.l == NullNode():
    if prev == NullNode():
      self.root = root.r
    elif value < prev.value:
      prev.l = root.r
    else:
        prev.r = root.r
  elif root.r == NullNode():
    if prev == NullNode():
      self.root = root.l
    elif value < prev.value:
      prev.l = root.l
    else:
      prev.r = root.l
  else:
    suc = root.successor().value
    root.value = suc
    self.delete(suc, root.r, root)

root.refresh_height()
root.rebalance()
```

7.4 Minimaler AVL-Baum

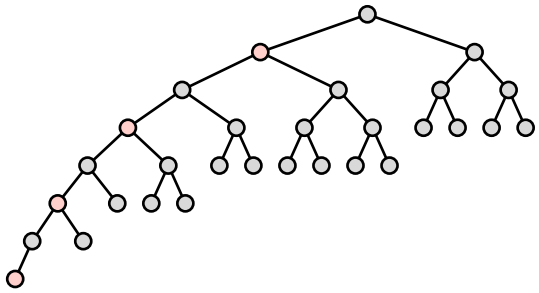
$$\mathcal{P}(h) = \begin{cases} 1 \text{ falls } h = 0 \\ 2 \text{ falls } h = 1 \\ 2 \cdot (\mathcal{P}(h-1) \cdot \mathcal{P}(h-2)) \end{cases}$$

$\mathcal{P}(5) = 4096$

8 RS- und B-Bäume

8.1 RS-Baum

Minimaler RS-Baum



Höhe RS-Baum

Der kürzestmögliche Pfad mit gegebener Black-Height besteht nur aus schwarzen Knoten. Der längstmögliche Pfad muss nach Eigenschaft 5 genauso viele schwarze Knoten haben, wird aber mit möglichst vielen roten Knoten aufgepumpt.

Nach RS-Baum Eigenschaft 4 kann nur jeder zweite Knoten auf diesem Pfad rot sein, weil rote Knoten keine roten Nachfolger haben dürfen.

Induktionsanfang $h = 1$

Ein Knoten v mit Höhe $h(v) = 1$ hat links und rechts das RS-Blatt. Die Pfadlängen sind beide 1. ✓

Induktionsannahme

Für alle Knoten v mit $h(v) = h(v') - 1$ gilt: Ein längster Pfad zu einem RS-Blatt ist höchstens doppelt so lang wie ein Kürzester.

Induktionsschritt $h - 1 \to h$

Fall 1: v ist rot. Der kürzeste Weg von v' ist um 1 kürzer als der von v, weil: v ist ein Nachfolger von v'. Wählen wir als anderen Nachfolger einen schwarzen Knoten, so können wir den ersten roten Knoten aus v „überspringen“.

Der längste Weg von v' ist um 1 länger als der von v, weil ein Knoten oben angehängt wurde.

$$\frac{s}{l} \leq 2 \Rightarrow \frac{s-1}{l+1} \leq 2$$

Fall 2: v ist schwarz. Der kürzeste Weg ist gleich, der Längste wieder um 1 länger.

$$\frac{s}{l} \leq 2 \Rightarrow \frac{s}{l+1} \leq 2 \quad \blacksquare$$

8.2 B-Baum Einfügen

8.3 B-Baum Löschen

8.4 B-Bäume konstruieren

9 Hashing

9.1 Prioritätswarteschlange

Ich implementiere die Priority Queue mit einem Heap.

Insert

Der Wert wird an das Ende des Heaps angehängt, und mit `heapify_btt` nach oben getragen, um die Heap-Eigenschaft wiederherzustellen. Laufzeit: $\Theta(\log n)$.

Get

Weil es ein Min-Heap ist, ist der minimale Wert der erste Eintrag. Laufzeit: $\Theta(1)$

Pop

Der erste Wert wird mit dem Letzten vertauscht, dann wird der letzte gelöscht. Die Heap-Eigenschaft wird mit `heapify_ttb` wiederhergestellt. Laufzeit: $\Theta(\log n)$

Dec

Der Eintrag an Stelle i wird aktualisiert, und mit `heapify_btt` nach oben getragen. Weil der neue Wert kleiner ist, muss er möglicherweise weiter oben stehen. Laufzeit: $\Theta(\log n)$

Alle Operationen laufen In-Situ. Die Platzkomplexität ist $\Theta(1)$.

9.2 Multiplikationsmethode

s	$h(s)$
61	700
62	318
63	936
64	554
65	172

9.3 Erwartungswert Kollisionen

$$P_k = \underbrace{\binom{n}{k}}_{(a)} \cdot \underbrace{\left(\frac{1}{m}\right)^k}_{(b)} \cdot \underbrace{\left(1 - \frac{1}{m}\right)^{n-k}}_{(c)}$$

(a) So viele Möglichkeiten gibt es, k Schlüssel aus einer Hashtabelle mit n Schlüsseln zu ziehen.
(b) Die Wahrscheinlichkeit, dass alle k Schlüssel auf die gleiche Stelle gehasht werden.
(c) Die Wahrscheinlichkeit, dass alle $n - k$ übrigen Schlüssel nicht auf diese Stelle gehasht werden.

Der Ausdruck P_k ist die erwartete Menge an Kollisionen zwischen k Werten.

9.4 Kollisionsauflösung

Lineares Probieren

```
h = self.hash(key)
for i in count():
  yield (h + i) % self._size
```

Quadratisches Probieren

```
h = self.hash(k)
for i in count():
  yield (h + i + 3*i**2) % self._size
```

Doppeltes Hashing

```
h_1 = self.hash(k)
h_2 = self.hash2(k)
for i in count():
  yield (h_1 + i * h_2) % self._size
```

Einfügen

Mit $m = 11$ und Werten 10, 22, 31, 4, 15, 28, 17, 88, 59.

	(a)	(b)	(c)
0	22	22	22
1	88		
2		88	59
3		17	17
4	4	4	4
5	15		15
6	28	28	28
7	17	59	88
8	59	15	
9	31	31	31
10	10	10	10

(a) Lineares Probieren
(b) Quadratisches Probieren
(c) Doppeltes Hashing

10 Textsuche

10.1 Skip-Liste

Einfügen

1. Höhe des Heads nach oben anpassen
2. Für alle Nachfolger des Heads, welche höchstens so hoch wie der neue Knoten sind, von oben nach unten den letzten Nachfolger, der kleiner als key ist, auf den neuen Knoten setzen.

Suchen

1. Am höchsten Nachfolger des Heads anfangen
2. Falls der Nachfolger kleiner oder gleich key ist, ihm folgen
3. Sonst eine Ebene runter gehen

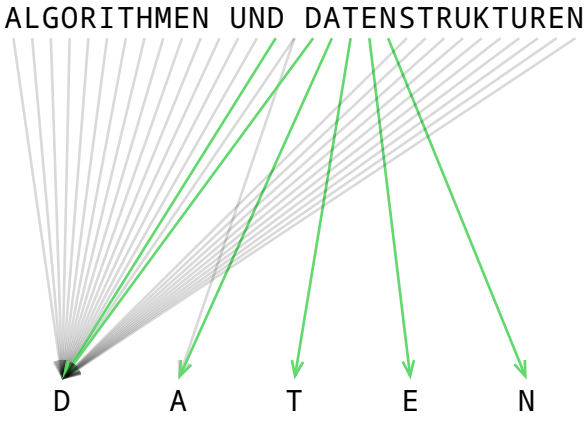
4. Wiederhole ab Schritt 2, bis key gefunden wurde oder height = –1.

Löschen

So wie einfügen, aber Nachfolger überbrücken.

10.2 Suche ausführen

Naive Suche



Boyer-Moore

```
ALGORITHMEN UND DATENSTRUKTUREN
DATEN
     DATEN
      DATEN
         DATEN
             DATEN
                DATEN
                 DATEN
                      DATEN
                        DATEN
```

11 Graphen

11.1 Adjazenzmatrix

Noah Jutz

CC 0

Algorithmen und Datenstrukturen

Appendix

1 Beweise 51
1.1 Stellenwertsystem 51
1.2 Grenzwert der Fibonacci-Folge 51
1.3 Kadane 51
2 Beispiele 52
2.1 AC-Laufzeit beim Inkrementieren . 52
2.2 Insertion Sort 52
2.3 Bubble Sort 54
2.4 Selection Sort 55
3 Code 56
3.1 MaxTeilSum-2d 56

1 Beweise

1.1 Stellenwertsystem ⤒

Zu zeigen

„Es gibt eine Stellenwertnotation für jede natürliche Zahl."

$\forall x \in \mathbb{N}, b \in \{2, 3, 4, ...\} : \exists\, x_n ... x_2 x_1 x_0$

mit $x_i \in \{0, 1, 2, ..., b - 1\}$

und $x = \sum_{i=0}^{m} a_i \cdot b^i$

Beweis

Induktionsanfang

$x = 1$

Die Stellenwertnotation von 1 ist für jede Basis 1.

$$1 = \sum_{i=0}^{0} \underbrace{x_0}_{=1} \cdot b^0 \checkmark$$

Induktionsschritt

$x \to x + 1$

Wenn es für x genau eine Stellenwertnotation $x_n ... x_2 x_1 x_0$ gibt, dann gibt es auch für $y = x + 1$ eine Stellenwertnotation $y_{n+1} ... y_2 y_1 y_0$.

Fall 1

Die niederwertigste Ziffer kann erhöht werden ($x_0 + 1 < b$)

Die Stellenwertnotation für $x + 1$ ist

$x_n ... x_2 x_1 (x_0 + 1) \checkmark$

Fall 2

Es gibt keinen höheren Ziffernwert ($x_0 + 1 = b$)

Weil der Stellenwert sich mit jeder Stelle um Faktor b erhöht, kann eine Zahl mit maximalem Ziffernwert $a_i = b - 1$ um 1 erhöht werden, indem man die vorangestellte Ziffer a_{i+1} erhöht (carry).

$$y = \sum_{i=0}^{n+1} y_i \cdot b^i$$

$$y_i = \begin{cases} x_i \text{ falls es keinen carry mehr gibt} \\ x_i + 1 \text{ falls } x_i + 1 < b \\ 0 \text{ sonst} \end{cases} \checkmark$$

Damit ist bewiesen, dass jede Stellenwertnotation um 1 erhöht werden kann, wodurch der Induktionsbeweis gilt. ■

1.2 Grenzwert der Fibonacci-Folge ⤒

Zu zeigen

Das Verhältnis zwei aufeinanderfolgender Fibonacci-Zahlen $\frac{f_{n+1}}{f_n}$ konvergiert gegen $\varphi \approx 1.618$.

Beweis

$$\begin{aligned} & f_n = f_{n-1} + f_{n-2} \\ \Leftrightarrow\ & \frac{f_n}{f_{n-1}} = \frac{f_{n-1}}{f_{n-1}} + \frac{f_{n-2}}{f_{n-1}} \\ \Leftrightarrow\ & \frac{f_{n+1}}{f_n} = 1 + \frac{f_n}{f_{n+1}} \\ \Leftrightarrow\ & x = 1 + \frac{1}{x} \\ \Leftrightarrow\ & x^2 = x + 1 \\ \Leftrightarrow\ & 0 = x^2 - x - 1 \\ \Leftrightarrow\ & x = \frac{1 \pm \sqrt{5}}{2} \end{aligned}$$

$$\varphi = \frac{1 + \sqrt{5}}{2} \approx 1.618 \ ■$$

1.3 Kadane ⤒

Zu zeigen

Der Algorithmus von Kadane zur Berechnung der maximalen Teilsumme eines Tupels $(x_1, x_2, x_3, ..., x_n)$ ist korrekt.

Beweis

Wir zeigen zunächst, dass nach jedem i-ten Schritt folgende Schleifeninvariante gilt: σ_i ist die maximale Teilsumme, welche an Stelle i endet.

Induktionsanfang

$i = 1$

Da $\sigma_0 = 0$ als das neutrale Element der Addition initialisiert wird, wird im dieser Wert ersten Schritt immer zu x_1. Die triviale Teilfolge der Länge 1 hat auch die maximale Teilsumme von $\sigma = x_1$.

$$\sigma_1 \coloneqq \max\{\sigma_0, \sigma_0 + x_1\} = \sigma_0 + x_1 = x_1 \checkmark$$

Induktionsschritt

$i \to i + 1$

Wenn $\sigma_i + x_i$ kleiner ist als x_i, und σ_i die maximale Teilsumme mit Ende i ist, dann gibt es keine Teilfolge vor x_i, welche die Teilsumme erhöhen würde.

$$\sigma_{i+1} = \max\{\sigma_i, \sigma_i + x_i\} \checkmark$$

Induktionsschluss

Weil σ_i die maximale Teilsumme, welche an Stelle i endet, ist, ist das Ergebnis einfach das Maximum aller σ_i.

$$\text{maxTeilSum} = \max_{0<i<n} \sigma_i \quad \blacksquare$$

2 Beispiele

2.1 AC-Laufzeit beim Inkrementieren ⤒

Sei A der Algorithmus, der eine binäre Zahl der Länge n um 1 erhöht.

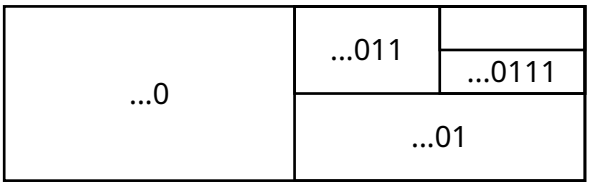
Es gibt $|\Sigma^n| = 2^n$ binäre Zahlen der From $\{0,1\}^n$.

$$T_A^{\text{AC}}(n) = \frac{1}{2^n} \sum_{x \in \{0,1\}^n} T(x)$$

Beim inkrementieren wird jede 1 am Ende zu einer Null, und anschließend wird der Carry vorangehängt. Die Laufzeit für ein gegebenes x ist also

$$T_A(x) = \max\{k \mid x_k \ldots x_2 x_1 x_0 = \{1\}^k\} + 1$$

Die Hälfte der Zahlen aus $\{0,1\}^n$ endet mit einer 1. Die Hälfte dieser Teilmenge endet wiederum mit zwei 1en, und so weiter.

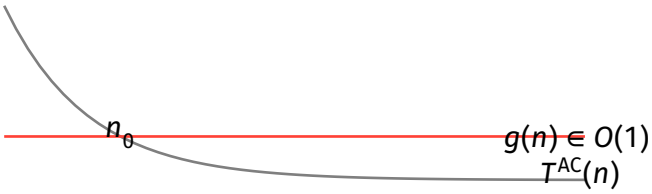


Daher ist die Summe aller Laufzeiten eine endliche geometrische Reihe mit der Basis $\frac{1}{2}$.

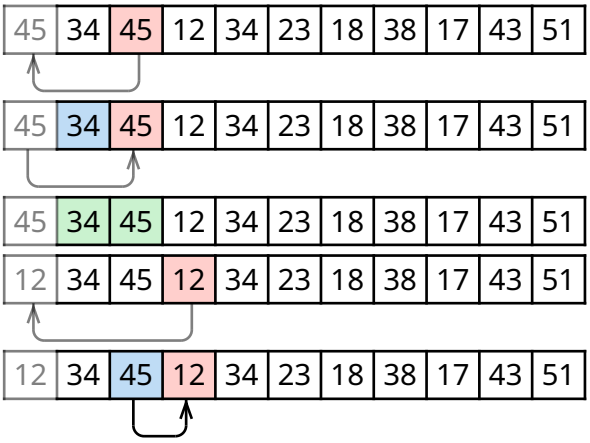
$$\sum_{x \in \{0,1\}^n} T_A(x) = 1 + \frac{1}{2} + \frac{1}{4} + \ldots + \frac{1}{2^n}$$
$$= \sum_{k=0}^{n} \left(\frac{1}{2}\right)^k$$

Diese Reihe konvergiert gegen 2. Die Average-Case-Laufzeit $T_A^{\text{AC}}(n)$ ist somit konstant.

$$T_A^{\text{AC}}(n) = \frac{1}{2^n} \cdot 2 \in O(1)$$



2.2 Insertion Sort ⤒



12	34	45	45	34	23	18	38	17	43	51
12	34	34	45	34	23	18	38	17	43	51
12	12	34	45	34	23	18	38	17	43	51
34	12	34	45	34	23	18	38	17	43	51
34	12	34	45	34	23	18	38	17	43	51
34	12	34	45	45	23	18	38	17	43	51
34	12	34	34	45	23	18	38	17	43	51
23	12	34	34	45	23	18	38	17	43	51
23	12	34	34	45	23	18	38	17	43	51
23	12	34	34	45	45	18	38	17	43	51
23	12	34	34	34	45	18	38	17	43	51
23	12	34	34	34	45	18	38	17	43	51
23	12	23	34	34	45	18	38	17	43	51
18	12	23	34	34	45	18	38	17	43	51
18	12	23	34	34	45	18	38	17	43	51
18	12	23	34	34	45	45	38	17	43	51
18	12	23	34	34	34	45	38	17	43	51
18	12	23	34	34	34	45	38	17	43	51
18	12	23	23	34	34	45	38	17	43	51
18	12	18	23	34	34	45	38	17	43	51
38	12	18	23	34	34	45	38	17	43	51

38	12	18	23	34	34	45	38	17	43	51
38	12	18	23	34	34	45	45	17	43	51
38	12	18	23	34	34	38	45	17	43	51
17	12	18	23	34	34	38	45	17	43	51
17	12	18	23	34	34	38	45	17	43	51
17	12	18	23	34	34	38	45	45	43	51
17	12	18	23	34	34	38	38	45	43	51
17	12	18	23	34	34	38	38	45	43	51
17	12	18	23	34	34	34	38	45	43	51
17	12	18	23	34	34	34	38	45	43	51
17	12	18	23	23	34	34	38	45	43	51
17	12	18	18	23	34	34	38	45	43	51
17	12	17	18	23	34	34	38	45	43	51
43	12	17	18	23	34	34	38	45	43	51
43	12	17	18	23	34	34	38	45	43	51
43	12	17	18	23	34	34	38	45	45	51
43	12	17	18	23	34	34	38	43	45	51
51	12	17	18	23	34	34	38	43	45	51
51	12	17	18	23	34	34	38	43	45	51
51	12	17	18	23	34	34	38	43	45	51

2.3 Bubble Sort

34	45	12	34	23	18	38	17	43	7
34	45	12	34	23	18	38	17	7	43
34	45	12	34	23	18	38	7	17	43
34	45	12	34	23	18	7	38	17	43
34	45	12	34	23	7	18	38	17	43
34	45	12	34	7	23	18	38	17	43
34	45	12	7	34	23	18	38	17	43
34	45	7	12	34	23	18	38	17	43
34	7	45	12	34	23	18	38	17	43
7	34	45	12	34	23	18	38	17	43
7	34	45	12	34	23	18	38	17	43
7	34	45	12	34	23	18	38	17	43
7	34	45	12	34	23	18	17	38	43
7	34	45	12	34	23	17	18	38	43
7	34	45	12	34	17	23	18	38	43
7	34	45	12	17	34	23	18	38	43
7	34	45	12	17	34	23	18	38	43
7	34	12	45	17	34	23	18	38	43
7	12	34

7	12	17	18	23	34	45	34	38	43
7	12	17	18	23	34	45	34	38	43
7	12	17	18	23	34	45	34	38	43
7	12	17	18	23	34	45	34	38	43
7	12	17	18	23	34	34	45	38	43
7	12	17	18	23	34	34	45	38	43
7	12	17	18	23	34	34	45	38	43
7	12	17	18	23	34	34	45	38	43
7	12	17	18	23	34	34	38	45	43
7	12	17	18	23	34	34	38	45	43
7	12	17	18	23	34	34	38	45	43
7	12	17	18	23	34	34	38	45	43
7	12	17	18	23	34	34	38	43	45
7	12	17	18	23	34	34	38	43	45
7	12	17	18	23	34	34	38	43	45
7	12	17	18	23	34	34	38	43	45

2.4 Selection Sort

34	45	12	34	23	18	38	17	43	7
7	45	12	34	23	18	38	17	43	34
7	45	12	34	23	18	38	17	43	34
7	12	45	34	23	18	38	17	43	34
7	12	45	34	23	18	38	17	43	34
7	12	17	34	23	18	38	45	43	34

7	12	17	34	23	18	38	45	43	34
7	12	17	18	23	34	38	45	43	34
7	12	17	18	23	34	38	45	43	34
7	12	17	18	23	34	38	45	43	34
7	12	17	18	23	34	38	45	43	34
7	12	17	18	23	34	38	45	43	34
7	12	17	18	23	34	38	45	43	34
7	12	17	18	23	34	38	45	43	34
7	12	17	18	23	34	34	45	43	38
7	12	17	18	23	34	34	45	43	38
7	12	17	18	23	34	34	38	43	45
7	12	17	18	23	34	34	38	43	45
7	12	17	18	23	34	34	38	43	45
7	12	17	18	23	34	34	38	43	45
7	12	17	18	23	34	34	38	43	45

3 Code

3.1 MaxTeilSum-2d ↑

```
import math
import numpy as np

def kadane(array):
    s = 0
    m = -math.inf
    for x in array:
        s = max(x, s + x)
        m = max(m, s)
    return m

def combinations(matrix):
    (n, _) = matrix.shape
    for i2 in range(n+1):
        for i1 in range(i):
            yield np.sum(matrix[i1:i2], axis=0)

def max_subarray_2d(matrix):
    return max(map(kadane, combinations(matrix)))

matrix = np.loadtxt("data.txt", dtype=int)
m = max_subarray_2d(matrix)
print(m)
```

Noah Jutz

Algorithmen und Datenstrukturen

Bibliographie

[1] „Fibonacci's Rabbits“. [Online]. Verfügbar unter: https://mathcenter.oxford.emory.edu/site/math125/fibonacciRabbits/

[2] Grant Sanderson, „Eigenvectors and Eigenvalues“, in *Essence of Linear Algebra*, Nr. 14. [Online]. Verfügbar unter: https://www.3blue1brown.com/lessons/eigenvalues

[3] Reducible, *Towers of Hanoi: A Complete Recursive Visualization*. [OnlineVideo]. Verfügbar unter: https://www.youtube.com/watch?v=rf6uf3jNjbo

[4] Jessica Su, „Rod cutting“. Zugegriffen: 25. September 2024. [Online]. Verfügbar unter: https://web.stanford.edu/class/archive/cs/cs161/cs161.1168/lecture12.pdf

[5] M. Ram Murty, „The Partition Function Revisited“. [Online]. Verfügbar unter: https://mast.queensu.ca/~murty/partition.pdf

[6] Back To Back SWE, *AVL Trees & Rotations (Self-Balancing Binary Search Trees)*. Zugegriffen: 13. November 2024. [OnlineVideo]. Verfügbar unter: https://www.youtube.com/watch?v=vRwi_UcZGjU

[7] David Galles, „AVL Tree Visualization“, Data Structure Visualizations. Zugegriffen: 13. November 2024. [Online]. Verfügbar unter: https://www.cs.usfca.edu/~galles/visualization/AVLtree.html

[8] Dr. Shun Yan Cheung, „Delete operations on AVL trees“. Zugegriffen: 14. November 2024. [Online]. Verfügbar unter: http://www.cs.emory.edu/~cheung/Courses/253/Syllabus/Trees/AVL-delete.html#case3

[9] Phillip Kirlin, „Deletion in a B-tree“. Zugegriffen: 19. November 2024. [Online]. Verfügbar unter: https://www.cs.rhodes.edu/~kirlinp/courses/db/f16/handouts/btrees-deletion.pdf

[10] Jerry Cain, „Hashing and HashMaps“, CS106X: Programming Abstractions in C++. Zugegriffen: 27. November 2024. [Online]. Verfügbar unter: https://web.stanford.edu/class/cs106x/res/handouts/22-Hashing-And-HashMaps.pdf

[11] Nitin Verma, „Primes in Division Method of Hashing“. Zugegriffen: 27. November 2024. [Online]. Verfügbar unter: https://mathsanew.com/articles/primes_in_division_method_hashing.pdf

[12] Clayton Cafiero, „Hash Tables: Quadratic Probing“. Zugegriffen: 25. November 2024. [Online]. Verfügbar unter: https://www.uvm.edu/~cbcafier/cs2240/content/10_probing/02_quadratic.html

[13] Dave Mount, „Randomized Search Structures: Skip Lists“, CMSC 420. Zugegriffen: 30. November 2024. [Online]. Verfügbar unter: https://www.cs.umd.edu/class/fall2020/cmsc420-0201/Lects/lect09-skip.pdf

[14] OpenDSA Project Contributors, „Skip Lists“. Zugegriffen: 30. November 2024. [Online]. Verfügbar unter: https://opendsa-server.cs.vt.edu/ODSA/Books/CS3/html/SkipList.html

[15] „Mathematical Induction“. Zugegriffen: 24. September 2024. [Online]. Verfügbar unter: https://en.wikipedia.org/wiki/Mathematical_induction#Induction_on_more_than_one_counter